

A. Prepare Multiple Nodes on Alibaba Cloud

A.1. Purchase Single Node on Alibaba Cloud

Step 1: Click <https://homenew.console.aliyun.com/home/dashboard/ProductAndService>, choose the ECS service,



Step 2: Create a new instance, choose pay as you go, choose ecs.c7.2xlarge, choose Ubuntu 16.04 64 bits, choose ESSD, choose no IPv4 address, choose your password Y0urpassword,



地域及可用区
华南3 (广州)

随机分配可用区A可用区B

如何选择地域
不同地域的实例之间内网互不相通; 选择靠近您客户的地域, 可降低网络时延、提高您客户的访问速度。

实例规格
实例规格族
场景配置选型
可购买的地域
申请按量高配

分类选型场景化选型

当前代所有代

筛选选择vCPU选择内存搜索规格名称, 如: ecs.g5.largeI/O 优化实例是否支持IPv6

架构X86 计算ARM 计算异构计算弹性裸金属服务器

分类通用型计算型内存型大数据型本地SSD高主频型共享型增强型最新推荐

规格族	实例规格	vCPU	内存	处理器主频/睿频	内网带宽	内网收发包	存储IOPS 基准/峰值	IPv6	参考价格	处理器型号
☐ 计算型 c7	ecs.c7.large	2 vCPU	4 GiB	~3.5 GHz	最高 10 Gbps	90 万 PPS	2 万/11 万	是	¥ 0.407698 /时	Intel Xeon(ice Lake) Platinum 8369B
☐ 计算型 c7	ecs.c7.xlarge	4 vCPU	8 GiB	~3.5 GHz	最高 10 Gbps	100 万 PPS	4 万/11 万	是	¥ 0.815397 /时	Intel Xeon(ice Lake) Platinum 8369B
☒ 计算型 c7	ecs.c7.2xlarge	8 vCPU	16 GiB	~3.5 GHz	最高 10 Gbps	160 万 PPS	5 万/11 万	是	¥ 1.630795 /时	Intel Xeon(ice Lake) Platinum 8369B

实例数量: 1 台配置费用: ¥ 1.673 /时下一步: 网络和安全组确认订单

Step 3: Double-check the order,

基础配置网络和安全组系统配置 (选填)分组设置 (选填)5 确认订单

⚠ 请注意!
您尚未设置实例登录凭证, 如需远程登录实例, 可返回前二步系统配置里配置登录凭证, 或创建后通过控制台“重置实例密码”操作完成设置。可参考 重置实例登录密码。

所选配置

基础配置
付费模式: 按量付费
购买数量: 1 台
地域及可用区: 广州 可用区A
镜像: Ubuntu 16.04 64位 (安全加固)
实例规格: 高主频计算型 hfc7 / ecs.hfc7.large (2vCPU 4GiB)
系统盘: ESSD云盘 40GiB, 随实例释放, PL1 (单盘IOPS性能上限5万)

网络和安全组
网络: 专有网络
公网带宽: 按使用流量 5Mbps
VPC: [默认]vpc-7xvpsr6ry0nisabitu8zf/ vpc-7xvpsr6ry0nisabitu8zf
安全组: 1). 默认安全组 (自定义端口)
交换机: [默认]vsw-7xvvd41ebxy57p6ahhxq6/ vsw-7xvvd41ebxy57p6ahhxq6/ 172.17.144.0/20

保存为启动模板生成Open API最佳实践脚本保存当前购买配置为ROS模板

使用期限☐ 设置自动释放服务时间 ECS实例将在您预约的时间点进行释放。实例释放后数据及IP地址不会被保留且无法找回, 请谨慎操作。

实例数量: 1 台配置费用: ¥ 0.555 /时公网流量费用: ¥ 0.800 /GB使用节省计划: ¥ 0.370 /时上一步: 分组设置创建实例

Step 4: Pay the order,



Step 5: Access the ECS instance via Alibaba Workbench, where the default username is root,

So far, the ECS instance cannot access the Internet because it has no IP address.

A.2. Purchase a NAT Gateway

Step 1: Click <https://vpc.console.aliyun.com/nat/cn-guangzhou/nats>, purchase 2 public IP addresses, create a NAT gateway,



Step 2: After purchasing 2 public IP addresses, we have 8.163.56.8 and 8.138.164.210,

弹性公网IP



A.3. Purchase Multiple Nodes on Alibaba Cloud

Step 1: Click <https://vpc.console.aliyun.com/vpc/cn-guangzhou/vpcs>, purchase another 24 ECS nodes,

专有网络

创建专有网络

模糊搜索

请输入关键词进行模糊搜索

×

Q

标签筛选

↓

↓

↓

实例ID/名称	状态	默认专有网络	路由表	交换机	云服务器	资源组	操作
vpc-7xviyvtbdf679qdy2l6r	可用	是	1	1	1	rg-acfmxzjqc76ccq 默认资源组	添加云产品 删除 云服务器 (ECS) 云数据库 (RDS)

每页显示 20 条 总共1条

choose pay as you go, choose ecs.c7.2xlarge, choose Ubuntu 16.04 64 bits, choose ESSD, choose no IPv4 address, choose your password YOurpassword,

公网 IP

☐ 分配公网 IPv4 地址

不为实例分配公网 IP 地址，如需访问公网，请配置并 绑定弹性公网 IP 地址，或者购买实例后升级实例的带宽，系统会自动为实例分配公网 IP

安全组

已有安全组 新建安全组

如何配置安全组

重新选择安全组

1) sg-7xviqssiy2djhqf5rzs (已有 1 个实例+辅助网卡，还可以加入 5999 个实例+辅助网卡)

请确保所选安全组开放包含 22 (Linux) 或者 3389 (Windows) 端口，否则无法远程登录ECS，前往设置

开启巨型帧

☐ 开启巨型帧

弹性网卡

主网卡

新建网卡 已有网卡

交换机 [默认]vsw-7xv5cj8r8xg2frivi113b ☒ 自动分配 IP 地址 ☒ 随实例释放 ☐ 源/目的检查

辅助网卡

新建网卡 已有网卡

+ 添加辅助弹性网卡 (0 / 1) 创建实例时只能附加 1 块弹性网卡，了解更多

IPv6

当前交换机 vsw-7xv5cj8r8xg2frivi113b 尚未开通 IPv6，立即开通

弹性网卡 | IPv6 (选项)

管理设置

登录凭证

密钥对 自定义密码 创建后设置

密钥对安全强度远高于常规自定义密码，可以避免暴力破解威胁，建议您使用密钥对创建实例

云盘

用途	类型	数量	容量	性能	设置
系统盘	ESSD 云盘	1	40 GIB	PL0 (单盘IOPS性能上限1万) IOPS 2280 吞吐 110.0MB/s	☒ 随实例释放 ☐ 加密

配置概要

保存为启动模板

购买实例数量 10

最小购买数量 10

当 ECS的 库存数量 < 最小购买数量，会创建失败；当 购买数量 >= 库存数量 >= 最小购买数量，会按照 库存数量 来创建

使用期限 ☐ 设置自动释放服务时间

价格概要

配置费用 ￥16.727959/时

合计金额 ￥16.727959/时

查看明细

点击确认下单即表示您已知悉并同意产品服务协议、服务等协议以及本页面中您勾选过的产品专属条款 (若有)

确认下单

A.4. Configure the NAT Gateway

Step 1: Click <https://vpc.console.aliyun.com/nat/cn-guangzhou/nats>, configure the NAT gateway, where the IP address 8.163.56.8 for DNAT and 8.138.164.210 for SNAT (DNAT for Internet users and SNAT for VM inside). Currently, we have 1 NAT, 2 Public IP addresses, and 11 ECS, our target is as follows:

For NAT (Public IP):

SNAT	DNAT
8.138.164.210	8.163.56.8

For ECS maps (Inside IP):

SNAT	DNAT
-	172.31.210.67
	172.31.210.70
	172.31.210.71
	172.31.210.72
	172.31.210.73
	172.31.210.74
	172.31.210.75
	172.31.210.76
	172.31.210.77
	172.31.210.78
	172.31.210.79
	172.31.210.82
	172.31.210.83
	172.31.210.84
	172.31.210.85
	172.31.210.86
	172.31.210.87
	172.31.210.88
	172.31.210.89
	172.31.210.90
	172.31.210.91
	172.31.210.92
	172.31.210.93
	172.31.210.94

← 创建SNAT条目

1. SNAT条目绑定单个弹性公网IP时，NAT网关访问同一个目的IP和端口的最大连接数为55000，您可以通过在规则中增加IP的方式提升并发能力；
2. SNAT条目配置后，如果ECS没有优先使用SNAT条目中的IP主动访问互联网，请参考[统一公网出口IP](#)来优化您的网络架构

可以配置任意网段，指定网段内的ECS通过配置的弹性公网IP访问互联网

条目名称 ?

DNAT Public IP and Port Number	DNAT Inside IP
8.163.56.8:9999	172.31.210.67
8.163.56.8:10000	172.31.210.70
8.163.56.8:10001	172.31.210.71
8.163.56.8:10002	172.31.210.72
8.163.56.8:10003	172.31.210.73
8.163.56.8:10004	172.31.210.74
8.163.56.8:10005	172.31.210.75
8.163.56.8:10006	172.31.210.76
8.163.56.8:10007	172.31.210.77
8.163.56.8:10008	172.31.210.78
8.163.56.8:10009	172.31.210.79
8.163.56.8:10010	172.31.210.82
8.163.56.8:10011	172.31.210.83
8.163.56.8:10012	172.31.210.84
8.163.56.8:10013	172.31.210.85
8.163.56.8:10014	172.31.210.86
8.163.56.8:10015	172.31.210.87
8.163.56.8:10016	172.31.210.88
8.163.56.8:10017	172.31.210.89
8.163.56.8:10018	172.31.210.90
8.163.56.8:10019	172.31.210.91
8.163.56.8:10020	172.31.210.92
8.163.56.8:10021	172.31.210.93
8.163.56.8:10022	172.31.210.94

← 创建DNAT条目

1.DNAT规则配置后，无法访问ECS，请优先排除ECS安全组配置问题

2.DNAT IP（所有端口）映射规则配置后，ECS没有优先使用SNAT IP主动访问互联网，请参考[统一公网出口IP](#)来优化您的网络架构

* 选择弹性公网IP地址

8.163.56.8 | eip-7xv4bffzma2w9a0jrfkck

* 选择私网IP地址

通过ECS或弹性网卡进行选择

172.31.210.67 | launch-advisor-20260331 | 主网卡

通过手动输入

* 端口设置

任意端口

具体端口

公网端口 9999

私网端口 22

协议类型 TCP

端口范围为1-65535，支持端口段，以"/"隔开，公私网端口段中的端口数量一致，公私网需同为端口或者端口段

端口范围为1-65535，支持端口段，以"/"隔开，公私网端口段中的端口数量一致，公私网需同为端口或者端口段

条目名称

DNAT_Node110/128

← 创建DNAT条目

1.DNAT规则配置后，无法访问ECS，请优先排除ECS安全组配置问题

2.DNAT IP（所有端口）映射规则配置后，ECS没有优先使用SNAT IP主动访问互联网，请参考[统一公网出口IP](#)来优化您的网络架构

* 选择弹性公网IP地址

8.163.56.8 | eip-7xv4bffzma2w9a0jrfkck

* 选择私网IP地址

通过ECS或弹性网卡进行选择

172.31.210.70 | launch-advisor-20260401 | 主网卡

通过手动输入

* 端口设置

任意端口

具体端口

公网端口 10000

私网端口 22

协议类型 TCP

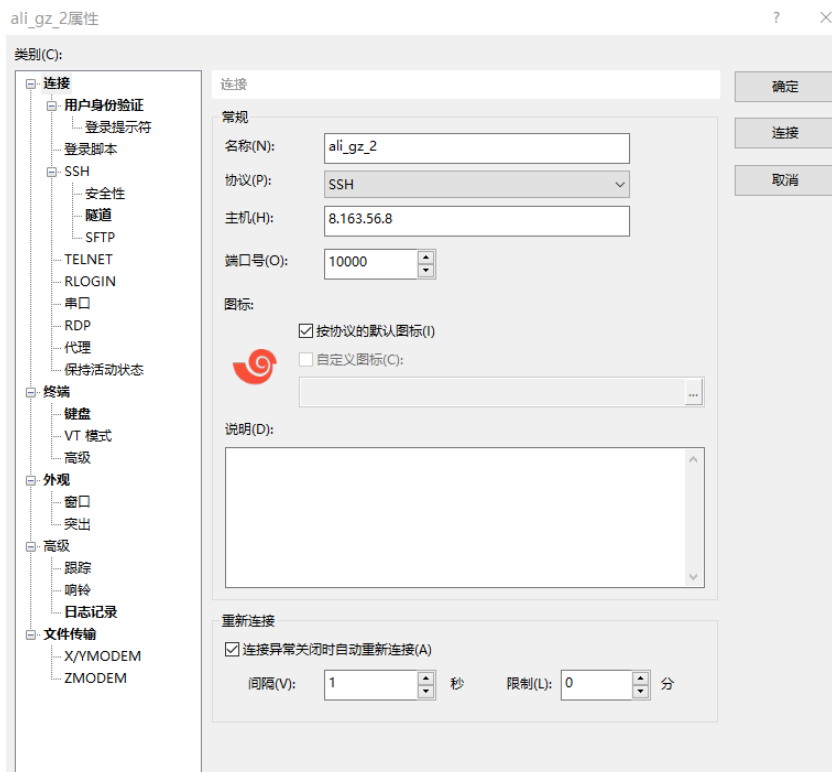
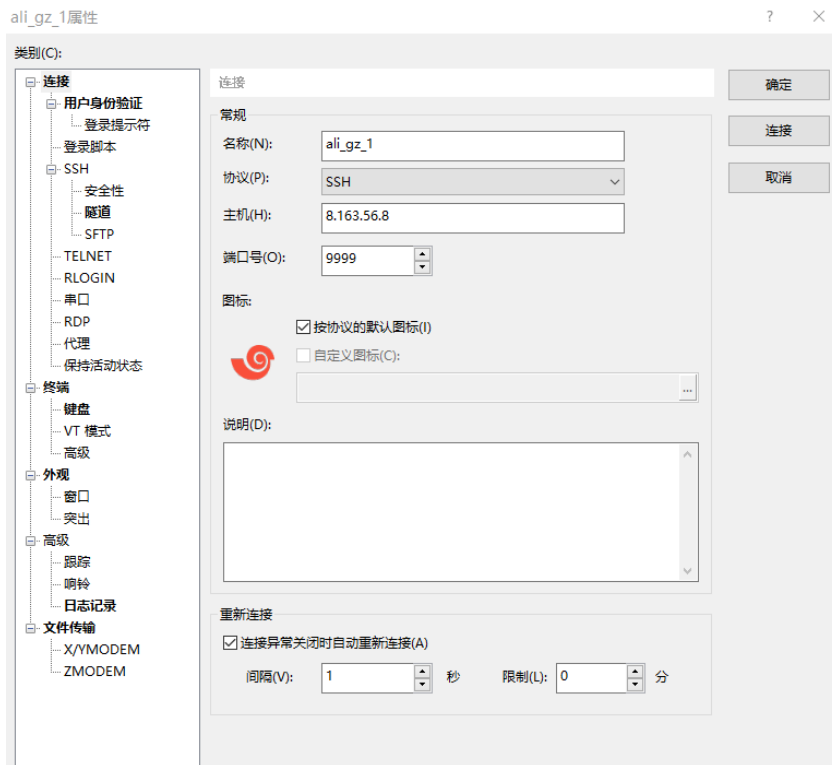
端口范围为1-65535，支持端口段，以"/"隔开，公私网端口段中的端口数量一致，公私网需同为端口或者端口段

端口范围为1-65535，支持端口段，以"/"隔开，公私网端口段中的端口数量一致，公私网需同为端口或者端口段

条目名称

DNAT_Node210/128

Access the inside IPs via Xshell,



A.5. Configure the password-free SSH

Step 1: The target is as follows,

Host with free-password SSH to	
172.31.210.67	-
	172.31.210.70
	172.31.210.71
	172.31.210.72
	172.31.210.73
	172.31.210.74
	172.31.210.75
	172.31.210.76
	172.31.210.77
	172.31.210.78
	172.31.210.79
	172.31.210.82
	172.31.210.83
	172.31.210.84
	172.31.210.85
	172.31.210.86
	172.31.210.87
	172.31.210.88
	172.31.210.89
	172.31.210.90
	172.31.210.91
	172.31.210.92
	172.31.210.93
	172.31.210.94

Step 2: Configure the 172.31.210.67,

```
$ ssh-keygen
```

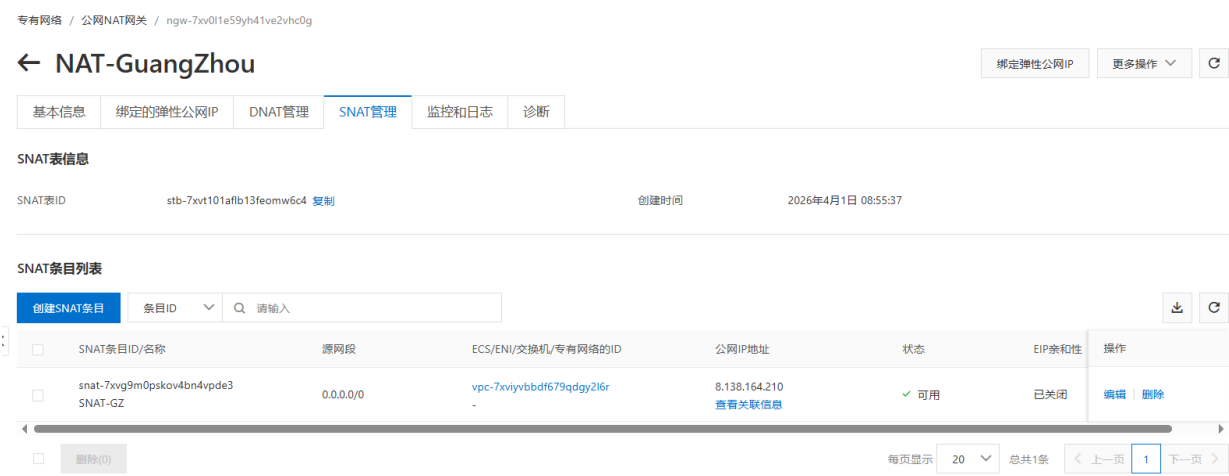
```
$ ssh root@172.31.210.70 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.71 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.72 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.73 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.74 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.75 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.76 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.77 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.78 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.79 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.82 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.83 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.84 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.85 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
```

```
$ ssh root@172.31.210.86 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.87 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.88 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.89 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.90 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.91 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.92 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.93 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.94 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
```

```
$ ssh 172.31.210.70 "echo 'hi'"
$ ssh 172.31.210.71 "echo 'hi'"
$ ssh 172.31.210.72 "echo 'hi'"
$ ssh 172.31.210.73 "echo 'hi'"
$ ssh 172.31.210.74 "echo 'hi'"
$ ssh 172.31.210.75 "echo 'hi'"
$ ssh 172.31.210.76 "echo 'hi'"
$ ssh 172.31.210.77 "echo 'hi'"
$ ssh 172.31.210.78 "echo 'hi'"
$ ssh 172.31.210.79 "echo 'hi'"
$ ssh 172.31.210.82 "echo 'hi'"
$ ssh 172.31.210.83 "echo 'hi'"
$ ssh 172.31.210.84 "echo 'hi'"
$ ssh 172.31.210.85 "echo 'hi'"
$ ssh 172.31.210.86 "echo 'hi'"
$ ssh 172.31.210.87 "echo 'hi'"
$ ssh 172.31.210.88 "echo 'hi'"
$ ssh 172.31.210.89 "echo 'hi'"
$ ssh 172.31.210.90 "echo 'hi'"
$ ssh 172.31.210.91 "echo 'hi'"
$ ssh 172.31.210.92 "echo 'hi'"
$ ssh 172.31.210.93 "echo 'hi'"
$ ssh 172.31.210.94 "echo 'hi'"
```

A.6. Free the NAT Gateway

Step 1: Click <https://vpc.console.aliyun.com/nat/cn-guangzhou/nats/ngw-7xv0l1e59yh41ve2vhc0g/snats>, delete SNAT,



Step 2: Click <https://vpc.console.aliyun.com/nat/cn-guangzhou/nats>, unbind public IP, free public IP,

Step 3: Click <https://homenew.console.aliyun.com/home/dashboard/ProductAndService>, delete ECS,

Step 4: Click <https://vpc.console.aliyun.com/nat/cn-guangzhou/nats>, delete NAT

B. Run Single-Node Fabric

B.1. Configure Each Node

Step 1: Configure the host names,

```
# sudo vi /etc/hosts
:set paste
i
```

```
172.31.210.67 cli # CLI

172.31.210.70 peer0.org1.example.com # Endorsing peer 1
172.31.210.71 peer0.org2.example.com # Endorsing peer 2
172.31.210.77 peer0.org3.example.com # Endorsing peer 3
172.31.210.78 peer0.org4.example.com # Endorsing peer 4
172.31.210.79 peer0.org5.example.com # Endorsing peer 5
172.31.210.82 peer0.org6.example.com # Endorsing peer 6
172.31.210.83 peer0.org7.example.com # Endorsing peer 7

172.31.210.84 peer0.org8.example.com # A Non-Endorsing Peer

172.31.210.72 orderer.example.com # Orderer 1
172.31.210.73 orderer2.example.com # Orderer 2
172.31.210.74 orderer3.example.com # Orderer 3
172.31.210.75 orderer4.example.com # Orderer 4
172.31.210.76 orderer5.example.com # Orderer 5
172.31.210.85 orderer6.example.com # Orderer 6
172.31.210.86 orderer7.example.com # Orderer 7
172.31.210.87 orderer8.example.com # Orderer 8
172.31.210.88 orderer9.example.com # Orderer 9
172.31.210.89 orderer10.example.com # Orderer 10
172.31.210.90 orderer11.example.com # Orderer 11
172.31.210.91 orderer12.example.com # Orderer 12
172.31.210.92 orderer13.example.com # Orderer 13
172.31.210.93 orderer14.example.com # Orderer 14
172.31.210.94 orderer15.example.com # Orderer 15
```

B.2. Install Dependencies on Each Node

Step 1: Install the prerequisites on each node,

```
// Prerequisite: Git
$ sudo apt-get update
$ sudo apt purge -y git
$ sudo apt-get install -y git // =1:2.7.4-0ubuntu1.9
$ git version // 2.7.4
```

```
$ export PATH=$PATH:/usr/lib/git-core
$ echo 'export PATH=$PATH:/usr/lib/git-core' >> ~/.bashrc
$ source ~/.bashrc
```

Step 2: Install the cURL on each node,

```
// Prerequisite: cURL
$ sudo apt-get install -y curl
$ curl --version // 7.47.0
```

Step 3: Install the wget on each node,

```
// Prerequisite: wget
$ sudo apt-get install -y wget=1.17.1-1ubuntu1.5
```

Step 4: Install Docker on each node,

```
// Prerequisite: Docker 17.06.2-ce or greater: Docker 17.12.0-ce
$ sudo apt-get purge -y docker-ce docker-ce-cli
$ wget https://download.docker.com/linux/ubuntu/gpg
$ sudo apt-key add gpg
$ echo 'deb [arch=amd64] https://download.docker.com/linux/ubuntu xenial stable' | sudo tee
/etc/apt/sources.list.d/docker.list
$ sudo apt-get update
$ apt-cache madison docker-ce
$ sudo apt-get install docker-ce=17.12.0~ce-0~ubuntu
$ sudo usermod -aG docker $USER
$ docker version
```

Step 5 (if outside the Great Wall): Install Docker Compose on each node,

```
// Prerequisite: Docker Compose 1.14.0 or greater: 1.21.0
$ sudo apt-get purge -y docker-compose
$ sudo curl -L https://github.com/docker/compose/releases/download/1.21.0/docker-compose-
`uname -s`-`uname -m` -o /usr/local/bin/docker-compose
$ sudo chmod +x /usr/local/bin/docker-compose
$ docker-compose version
```

Step 5 (if within the Great Wall): Install Docker Compose on each node,

```
// Prerequisite: Docker Compose 1.14.0 or greater: 1.21.0
$ sudo apt-get purge -y docker-compose
$ mkdir downloads
$ cd $HOME/downloads
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/docker-compose
// $ scp docker-compose root@172.31.210.82:$HOME/downloads
```

```
// $ scp docker-compose root@172.31.210.83:$HOME/downloads
// $ scp docker-compose root@172.31.210.84:$HOME/downloads
// $ scp docker-compose root@172.31.210.85:$HOME/downloads
// $ scp docker-compose root@172.31.210.86:$HOME/downloads
// $ scp docker-compose root@172.31.210.87:$HOME/downloads
// $ scp docker-compose root@172.31.210.88:$HOME/downloads
// $ scp docker-compose root@172.31.210.89:$HOME/downloads
// $ scp docker-compose root@172.31.210.90:$HOME/downloads
// $ scp docker-compose root@172.31.210.91:$HOME/downloads
// $ scp docker-compose root@172.31.210.92:$HOME/downloads
// $ scp docker-compose root@172.31.210.93:$HOME/downloads
// $ scp docker-compose root@172.31.210.94:$HOME/downloads

$ sudo chmod +x docker-compose
$ sudo mv docker-compose /usr/local/bin
$ docker-compose version
```

Step 6 (if outside the Great Wall): Install Golang on each node,

```
// Prerequisite: Go v1.14.8
$ cd $HOME/downloads
$ wget https://golang.org/dl/go1.14.8.linux-amd64.tar.gz
// $ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.82:$HOME/downloads

$ tar -xzf go1.14.8.linux-amd64.tar.gz
$ sudo rm -rf /usr/local/go
$ sudo mv go /usr/local
$ mkdir -p $HOME/go
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
$ echo 'export GOPATH=$HOME/go' >> ~/.bashrc
$ source ~/.bashrc
$ go version // go1.14.8 linux/amd64
```

Step 6 (if within the Great Wall): Install the Golang on each node,

```
// Prerequisite: Go v1.14.8
$ cd $HOME/downloads
$ ... // download go1.14.8.linux-amd64.tar.gz on node one, e.g., via winscp,
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.70:$HOME/downloads
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.71:$HOME/downloads
$ ...
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.79:$HOME/downloads

$ tar -xzf go1.14.8.linux-amd64.tar.gz
$ sudo mv go /usr/local
$ mkdir -p $HOME/go
```

```
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
$ echo 'export GOPATH=$HOME/go' >> ~/.bashrc
$ source ~/.bashrc
$ go version // go1.14.8 linux/amd64
```

Step 7: Install Node.js on each node,

```
// Prerequisite: Node.js v12.20.1 or higher
$ sudo apt-get purge -y node
$ sudo apt-get purge -y nodejs
$ cd $HOME/downloads
$ wget https://registry.npmirror.com/-/binary/node/v12.22.12/node-v12.22.12-linux-x64.tar.xz
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.82:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.83:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.84:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.85:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.86:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.87:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.88:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.89:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.90:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.91:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.92:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.93:$HOME/downloads
$ scp node-v12.22.12-linux-x64.tar.xz root@172.31.210.94:$HOME/downloads

$ sudo tar -xJf node-v12.22.12-linux-x64.tar.xz
$ mv node-v12.22.12-linux-x64 node-v12.22.12
$ sudo mv node-v12.22.12 /usr/local
$ echo 'export PATH=$PATH:/usr/local/node-v12.22.12/bin' >> ~/.bashrc
$ source ~/.bashrc
$ node -v
$ npm -v
$ npx -v
```

Step 8: Install the NTP on each node,

```
$ sudo apt-get install ntp
$ sudo service ntp start
```

B.3. Install Fabric 2.2.0

Step 1 (if outside the Great Wall): Install Fabric 2.2.0, including Fabric Samples on each node,

```
$ cd $HOME
```



```
$ sudo rm -rf fabric-samples
$ curl -sSL https://bit.ly/2ysbOFE | bash -s -- 2.2.0
```

Step 1 (if within the Great Wall): Install Fabric 2.2.0, including Fabric Samples on each node,

```
$ cd $HOME
$ sudo rm -rf fabric-samples
$ sudo apt-get install unzip
$ cd $HOME
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/fabric-v2.2.0.zip

$ scp fabric-v2.2.0.zip root@172.31.210.82:$HOME/downloads
$ scp fabric-v2.2.0.zip root@172.31.210.83:$HOME/downloads
$ scp fabric-v2.2.0.zip root@172.31.210.84:$HOME/downloads
$ scp fabric-v2.2.0.zip root@172.31.210.85:$HOME/downloads
$ scp fabric-v2.2.0.zip root@172.31.210.86:$HOME/downloads
$ scp fabric-v2.2.0.zip root@172.31.210...:$HOME/downloads
$ scp fabric-v2.2.0.zip root@172.31.210.94:$HOME/downloads

$ unzip fabric-v2.2.0.zip
$ cd fabric-v2.2.0/scripts
```

In bootstrap.sh -> "pullBinaries()",

```
download "${BINARY_FILE}"
"https://github.com/hyperledger/fabric/releases/download/v${VERSION}/${BINARY_FILE}"
download "${CA_BINARY_FILE}" "https://github.com/hyperledger/fabric-
ca/releases/download/v${CA_VERSION}/${CA_BINARY_FILE}"
```

Replace with the two lines,

```
download "${BINARY_FILE}"
"https://gitee.com/joe320653/fabric/attach_files/842230/download/${BINARY_FILE}"
download "${CA_BINARY_FILE}" "https://gitee.com/joe320653/fabric-
ca/attach_files/842289/download/${CA_BINARY_FILE}"
```

In bootstrap.sh -> "cloneSamplesRepo()",

```
git clone -b master https://github.com/hyperledger/fabric-samples.git && cd fabric-samples && git
checkout v${VERSION}
```

Replace with the two lines,

```
git clone -b master https://gitee.com/joe320653/fabric-samples.git && cd fabric-samples && git
checkout v${VERSION}
```

Step 2: Install docker images and fabric-samples on each node,

```
$ cd $HOME/fabric-v2.2.0/scripts
$ chmod 777 *.sh

$ // configure the docker container service within the Great Wall
$ cd $HOME
$ sudo mkdir -p /etc/docker
$ sudo tee /etc/docker/daemon.json <<-'EOF'
{
  "registry-mirrors": ["https://2htzxp9d.mirror.aliyuncs.com"]
}
EOF
$ sudo systemctl daemon-reload
$ sudo systemctl restart docker

$ cd $HOME/fabric-v2.2.0/scripts
$ sudo ./bootstrap.sh
$ mv fabric-samples $HOME
```

B.4. *Run Test-Network*

Step 1 (if outside the Great Wall): Run test network on each node,

```
$ cd $HOME/fabric-samples/test-network
$ ./network.sh up
$ ./network.sh createChannel
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
$ ./network.sh down
```

The success information is as follows,

```
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2
MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2
MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

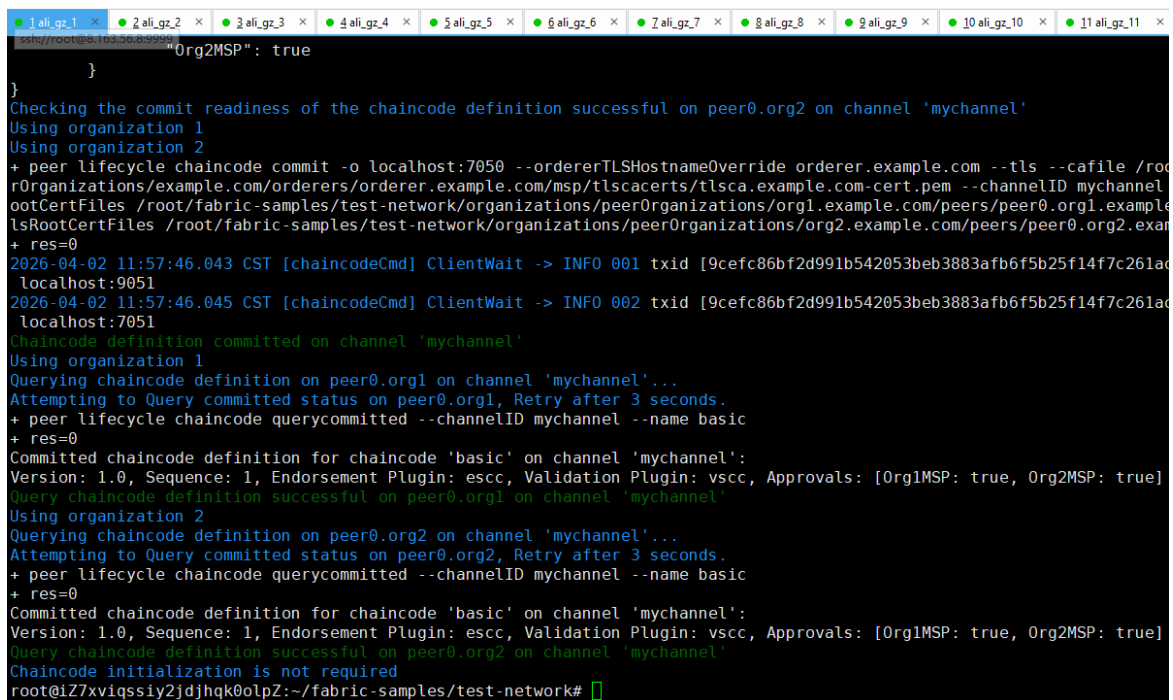
Step 1 (if within the Great Wall): Run test network on each node,

Set the Golang Proxy,

```
$ go env -w GOPROXY=https://goproxy.cn
```

Run the test network,

```
$ cd $HOME/fabric-samples/test-network
$ ./network.sh up
$ ./network.sh createChannel
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
$ ./network.sh down
```



```
1 ali_gz_1 x 2 ali_gz_2 x 3 ali_gz_3 x 4 ali_gz_4 x 5 ali_gz_5 x 6 ali_gz_6 x 7 ali_gz_7 x 8 ali_gz_8 x 9 ali_gz_9 x 10 ali_gz_10 x 11 ali_gz_11 x
root@i7xviqssiy2jdhqk0lpZ:~/fabric-samples/test-network# ./network.sh up
"Org2MSP": true
}
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /root/.fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tlsRootCertFiles /root/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tlsRootCertFiles
+ res=0
2026-04-02 11:57:46.043 CST [chaincodeCmd] ClientWait -> INFO 001 txid [9cefc86bf2d991b542053beb3883afb6f5b25f14f7c261ad0b5b25f14f7c261ad]
localhost:9051
2026-04-02 11:57:46.045 CST [chaincodeCmd] ClientWait -> INFO 002 txid [9cefc86bf2d991b542053beb3883afb6f5b25f14f7c261ad0b5b25f14f7c261ad]
localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
root@i7xviqssiy2jdhqk0lpZ:~/fabric-samples/test-network#
```

1. Run Multi-Nodes Fabric (Exp 1: 1 CLI + 2 Peers + 1 Orderer)

1.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i2/xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat tick:1 max inflight blocks:5 snapshot interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderer1 sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i2/xviqssiy2jdsfcldp22:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ...

```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
4a207f175833   hyperledger/fabric-orderer:2.2.0   "orderer"               8 seconds ago Up 8 seconds   0.0.0.0:7050->7050/tcp              orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
38fb1129b2ae   hyperledger/fabric-peer:2.2.0      "peer node start"       2 seconds ago Up 1 second    0.0.0.0:7051->7051/tcp              peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer2.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
e88f655db1d9   hyperledger/fabric-peer:2.2.0      "peer node start"       2 seconds ago Up 1 second    7051/tcp, 0.0.0.0:8051->8051/tcp    peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS    NAMES
30e6fbad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago Up 1 second        cli
```

1.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`  
# cd scripts  
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# docker ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES  
38e67bad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago Up 1 second     
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# docker exec -it cli bash  
bash-5.0# cd scripts  
bash-5.0# ./step1.sh  
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)  
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized  
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized  
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized  
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized  
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized  
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized  
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received blocks: 0  
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:38.953 UTC [cli.common] readBlock -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
11ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"nGmycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at  
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f50ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f0eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffbf97b289c38f53946f397b242b6d3ff591f2ba4f48aed23911f3] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
200tps  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
bash-5.0#
```

1.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read pem.py
$ vi prepare tls peer0 org1.sh
$ vi prepare tls peer0 org2.sh
$ ./all.sh

2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare wallet user1 org1.py
$ vi prepare wallet user1 org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare sdk peer0 org1.sh
$ vi prepare sdk peer0 org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code

https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare wallet user1 org1.py
```



```
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
root@i27xviqssiy2jdsfcldbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507

// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs

root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178

$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase

root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfcdpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfcdpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// Get block size
# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

2. Run Multi-Nodes Fabric (Exp 2: 1 CLI + 2 Peers + 5 Orderers)

2.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" e
ction ticks:10 heartbeat tick:1 max inflight blocks:5 snapshot_interval size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first#
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk 100% 241 0.2KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
orderer.example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
server.key 100% 241 0.2KB/s 00:0
server.crt 100% 916 0.9KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
client.crt 100% 814 0.8KB/s 00:0
client.key 100% 241 0.2KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@i27xviqssiy2jdsfcldpC2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

```
$ ./orderer2.sh
```

```
root@i27xviqssiy2jdsfcldpbwZ:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done
```

```
$ ./orderer3.sh
```

```
root@i27xviqssiy2jdsfcldpbZ:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done
```

\$ **./orderer4.sh**

```
root@i27xviqssiy2jdsfclpc3Z:~/fabric-samples/demo-first# ./orderer4.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer4.example.com" with default driver
Creating orderer4.example.com ... done
```

\$ **./orderer5.sh**

```
root@i27xviqssiy2jdsfclpc0Z:~/fabric-samples/demo-first# ./orderer5.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer5.example.com" with default driver
Creating orderer5.example.com ... done
```

\$ docker ps

Step 6 (Run): Peer1 sets up the Docker container,

\$ cd \$HOME/fabric-samples/demo-first

\$ **./peer1.sh**

```
root@i27xviqssiy2jdsfclpc0Z:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

\$ docker ps

Step 7 (Run): Peer2 sets up the Docker container,

\$ cd \$HOME/fabric-samples/demo-first

\$ **./peer2.sh**

```
root@i27xviqssiy2jdsfclpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

\$ docker ps

Step 8 (Run): CLI sets up the Docker container,

\$ cd \$HOME/fabric-samples/demo-first

\$ **./cli.sh**

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

\$ docker ps

2.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash
```

// Environment Configuration for `peer0.org1.example.com:7051`

cd scripts

./step1.sh // All peers join the channel

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto          scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh      step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./step2.sh // All peers join the chaincode

2.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read\_pem.py
$ vi prepare\_tls\_peer0\_org1.sh
$ vi prepare\_tls\_peer0\_org2.sh
$ ./all.sh
```

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare\_wallet\_user1\_org1.py
$ vi prepare\_wallet\_user1\_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
```

```
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare\_sdk\_peer0\_org1.sh
$ vi prepare\_sdk\_peer0\_org2.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare\_wallet\_user1\_org1.py
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
```

```
$ ls result -l
```

```
root@i27xviqssly2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssly2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
```

```
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssly2jdjhqk80lpz:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer' | awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@iZ7xviqssiy2jdsfcdpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@iZ7xviqssiy2jdsfcdpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// Get block size
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```


Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./clean.sh
```

3. Run Multi-Nodes Fabric (Exp 3: 1 CLI + 5 Peers + 5 Orderers)

3.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
172.31.210.77 peer0.org3.example.com
172.31.210.78 peer0.org4.example.com
172.31.210.79 peer0.org5.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
$ vi peer0-org3.yaml
$ vi peer0-org4.yaml
$ vi peer0-org5.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp
```

```
$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@127xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.019 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk 100% 241 0.2KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
orderer.example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
server.key 100% 241 0.2KB/s 00:0
server.crt 100% 916 0.9KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
client.crt 100% 814 0.8KB/s 00:0
client.key 100% 241 0.2KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

$ ./orderer2.sh

$ ./orderer3.sh

$ ./orderer4.sh
```

```
$ ./orderer5.sh
```

```
$ docker ps
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer1.sh
```

```
$ docker ps
```

Step 7 (Run): Other peers set up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer2.sh
```

```
$ ./peer3.sh
```

```
$ ./peer4.sh
```

```
$ ./peer5.sh
```

```
$ docker ps
```

Step 8 (Run): CLI sets up the docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./cli.sh
```

```
root@i2/xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh  
Creating cli ... done
```

```
$ docker ps
```

3.2. Start the Network

Step 1 (Run): CLI (of peer0.org1) connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
```

```
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`
```

cd scripts

./step1.sh // All peers join the channel

```
root@i27xviqssiy2jdjhqk80lpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
38e6fbad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago Up 1 second   ports        cli
root@i27xviqssiy2jdjhqk80lpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.650 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:37.652 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.049 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./step2.sh // All peers join the chaincode

```
2026-04-20 09:35:34.485 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [601b6ba90405a62832c471b05eb27696a5ab493c4b38ec245009af7c9626ee6] committed with status (VALID)
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true,
    "Org3MSP": true,
    "Org4MSP": true,
    "Org5MSP": true
  }
}
2026-04-20 09:35:35.665 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org2.example.com:7051
2026-04-20 09:35:35.667 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org1.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 003 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org5.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 004 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org4.example.com:7051
2026-04-20 09:35:35.670 UTC [chaincodeCmd] ClientWait -> INFO 005 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org3.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [0968d089aa85538bcac17bd89ac2438435d186162a5915195bed3c374dcebf67] committed with status (VALID)
peer0.org1.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-20 09:35:37.970 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [207d83ba4cab05dc9c6e742b66aba3cab5fdae5e7237cba5a93d9967945a9bc] committed with status (VALID)
peer0.org1.example.com:7051
2026-04-20 09:35:37.970 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-20 09:35:39.054 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [891f28608f3b23220f2e49a556a69afc5395fe5ca58300c0c516f4aae1e12c2] committed with status (VALID)
peer0.org2.example.com:7051
2026-04-20 09:35:39.055 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
3000tps
bash-5.0#
```

3.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

exit

\$ mkdir -p \$HOME/fabric-samples/demo-first/workload

\$ mkdir -p \$HOME/fabric-samples/demo-first/workload/tls

\$ mkdir -p \$HOME/fabric-samples/demo-first/workload/wallet

\$ mkdir -p \$HOME/fabric-samples/demo-first/workload/result

\$ cd \$HOME/fabric-samples/demo-first/workload/tls

\$ vi [read_pem.py](#)

\$ vi [prepare_tls_peer0_org1.sh](#)

```
$ vi prepare\_tls\_peer0\_org2.sh
$ vi prepare\_tls\_peer0\_org3.sh
$ vi prepare\_tls\_peer0\_org4.sh
$ vi prepare\_tls\_peer0\_org5.sh
$ ./all.sh
```

```
2020-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare\_wallet\_user1\_org1.py
$ vi prepare\_wallet\_user1\_org2.py
$ vi prepare\_wallet\_user1\_org3.py
$ vi prepare\_wallet\_user1\_org4.py
$ vi prepare\_wallet\_user1\_org5.py
$ ./all.sh
```

```
$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ mkdir sdk-peer0-org3
$ mkdir sdk-peer0-org4
$ mkdir sdk-peer0-org5
$ vi invoke.js
$ vi prepare\_sdk\_peer0\_org1.sh
$ vi prepare\_sdk\_peer0\_org2.sh
$ vi prepare\_sdk\_peer0\_org3.sh
$ vi prepare\_sdk\_peer0\_org4.sh
$ vi prepare\_sdk\_peer0\_org5.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#), [User1@org2.example.com](#), ..., and [User1@org5.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare\_wallet\_user1\_org1.py
$ python prepare\_wallet\_user1\_org2.py
$ python prepare\_wallet\_user1\_org3.py
$ python prepare\_wallet\_user1\_org4.py
$ python prepare\_wallet\_user1\_org5.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
```

```
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
```

```
$ ls result -l
```

```
root@i27xviqssiy2jdsfcldbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
```

```
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ python readthroughput_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
```

```
$ python readdelay_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.001080016583933
CDF with the probability of 90% 0.002080009346088
CDF with the probability of 95% 0.003080002098083
```

```
// Orderer analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ sudo apt-get install python-numpy
```

```
//$ docker logs containername >& ordererlog
```

```
$ docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog
```

```
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
```

```
$ python readthroughput_p2.py // rate of generating blocks
```

```

root@i27xviqssiy2jdsfclpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]

```

```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py

```

```

root@i27xviqssiy2jdsfclpc2Z:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

```

```

// Get block size
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```

Step 6 (Run): All nodes run the clean procedure,

```

$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh

```


4. Run Multi-Nodes Fabric (Exp 4: 7 CLI + 1 Peers + 15 Orderers)

4.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli # CLI
```

```
172.31.210.70 peer0.org1.example.com # Endorsing peer 1
172.31.210.71 peer0.org2.example.com # Endorsing peer 2
172.31.210.77 peer0.org3.example.com # Endorsing peer 3
172.31.210.78 peer0.org4.example.com # Endorsing peer 4
172.31.210.79 peer0.org5.example.com # Endorsing peer 5
172.31.210.82 peer0.org6.example.com # Endorsing peer 6
172.31.210.83 peer0.org7.example.com # Endorsing peer 7
```

```
172.31.210.84 peer0.org8.example.com # A Non-Endorsing Peer
```

```
172.31.210.72 orderer.example.com # Orderer 1
172.31.210.73 orderer2.example.com # Orderer 2
172.31.210.74 orderer3.example.com # Orderer 3
172.31.210.75 orderer4.example.com # Orderer 4
172.31.210.76 orderer5.example.com # Orderer 5
172.31.210.85 orderer6.example.com # Orderer 6
172.31.210.86 orderer7.example.com # Orderer 7
172.31.210.87 orderer8.example.com # Orderer 8
172.31.210.88 orderer9.example.com # Orderer 9
172.31.210.89 orderer10.example.com # Orderer 10
172.31.210.90 orderer11.example.com # Orderer 11
172.31.210.91 orderer12.example.com # Orderer 12
172.31.210.92 orderer13.example.com # Orderer 13
172.31.210.93 orderer14.example.com # Orderer 14
172.31.210.94 orderer15.example.com # Orderer 15
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
$ vi peer0-org3.yaml
```

```

$ vi peer0-org4.yaml
$ vi peer0-org5.yaml
$ vi peer0-org6.yaml
$ vi peer0-org7.yaml
$ vi peer0-org8.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml
$ vi orderer6.yaml
$ vi orderer7.yaml
$ vi orderer8.yaml
$ vi orderer9.yaml
$ vi orderer10.yaml
$ vi orderer11.yaml
$ vi orderer12.yaml
$ vi orderer13.yaml
$ vi orderer14.yaml
$ vi orderer15.yaml

$ vi cli.yaml

```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```

$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again

```

Step 3 (Run): CLI runs the initial block generation,

```

$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh

```

```

root@i2/xviqssiy2jdjhqk80lp2:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election
ck10 heartbeat ticks:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./scp.sh
```

```
priv_sk 100% 241 0.2KB/s 00:0  
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0  
orderer.example.com-cert.pem 100% 769 0.8KB/s 00:0  
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0  
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0  
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0  
priv_sk 100% 241 0.2KB/s 00:0  
ca.crt 100% 847 0.8KB/s 00:0  
server.key 100% 241 0.2KB/s 00:0  
server.crt 100% 916 0.9KB/s 00:0  
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0  
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0  
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0  
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0  
priv_sk 100% 241 0.2KB/s 00:0  
ca.crt 100% 847 0.8KB/s 00:0  
client.crt 100% 814 0.8KB/s 00:0  
client.key 100% 241 0.2KB/s 00:0  
priv_sk 100% 241 0.2KB/s 00:0  
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./orderer1.sh
```

```
$ ./orderer2.sh
```

```
$ ./orderer3.sh
```

```
$ ./orderer4.sh
```

```
$ ./orderer5.sh
```

```
$ ./orderer6.sh
```

```
$ ./orderer7.sh
```

```
$ ./orderer8.sh
```

```
$ ./orderer9.sh
```

```
$ ./orderer10.sh
```

```
$ ./orderer11.sh
```

```
$ ./orderer12.sh
```

```
$ ./orderer13.sh
```

```
$ ./orderer14.sh
```

```
$ ./orderer15.sh
```

```
$ docker ps
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./peer1.sh  
  
$ docker ps
```

Step 7 (Run): Other peers set up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./peer2.sh  
  
$ ./peer3.sh  
  
$ ./peer4.sh  
  
$ ./peer5.sh  
  
$ ./peer6.sh  
  
$ ./peer7.sh  
  
$ ./peer8.sh  
  
$ docker ps
```

Step 8 (Run): CLI sets up the docker container,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./cli.sh  
  
root@f27xviqssiy2jdjhqk00lpz:~/fabric-samples/demo-first# ./cli.sh  
Creating cli ...  
  
$ docker ps
```

4.2. Start the Network

Step 1 (Run): CLI (of peer0.org1) connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
```

```
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`
```

```
# cd scripts
```

```
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhq80lp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
30e6fbad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"             2 seconds ago Up 1 second   peer0.org1
root@i27xviqssiy2jdjhq80lp2:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.650 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.077 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
2026-04-20 09:35:34.485 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [601b6ba90405a662832c471b05eb27696a5ab493c4b38ec2450099af7c9626ee6] committed with status (VALID)
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true,
    "Org3MSP": true,
    "Org4MSP": true,
    "Org5MSP": true
  }
}
2026-04-20 09:35:35.665 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org2.example.com:7051
2026-04-20 09:35:35.667 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org1.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 003 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org5.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 004 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org4.example.com:7051
2026-04-20 09:35:35.670 UTC [chaincodeCmd] ClientWait -> INFO 005 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
peer0.org3.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [0968d89aa85538bcac17bd89ac2438435d186162a5915195bed3c374dceb67] committed with status (VALID)
peer0.org1.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-20 09:35:37.970 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [207d83ba4cab05dc9c6e742b66aba3cab5fdae5e7237cba5a93d996b7945a9bc] committed with status (VALID)
peer0.org1.example.com:7051
2026-04-20 09:35:37.970 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-20 09:35:39.054 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [891f28608f3b23220f2e49a556a69afc5395fe5ca58300c0c516f4aae1e12c2] committed with status (VALID)
peer0.org2.example.com:7051
2026-04-20 09:35:39.055 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
3000tps
bash-5.0#
```

4.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
```

```
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ vi prepare_tls_peer0_org3.sh
$ vi prepare_tls_peer0_org4.sh
$ vi prepare_tls_peer0_org5.sh
$ vi prepare_tls_peer0_org6.sh
$ vi prepare_tls_peer0_org7.sh
$ vi prepare_tls_peer0_org8.sh
$ ./all.sh
```

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i7xviqssi2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i7xviqssi2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i7xviqssi2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ vi prepare_wallet_user1_org3.py
$ vi prepare_wallet_user1_org4.py
$ vi prepare_wallet_user1_org5.py
$ vi prepare_wallet_user1_org6.py
$ vi prepare_wallet_user1_org7.py
$ vi prepare_wallet_user1_org8.py
$ ./all.sh
```

```
$ cd $HOME/fabric-samples/demo-first/workload
```

```
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ mkdir sdk-peer0-org3
$ mkdir sdk-peer0-org4
$ mkdir sdk-peer0-org5
$ mkdir sdk-peer0-org6
$ mkdir sdk-peer0-org7
$ mkdir sdk-peer0-org8
$ vi invoke.js

$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ vi prepare_sdk_peer0_org3.sh
$ vi prepare_sdk_peer0_org4.sh
$ vi prepare_sdk_peer0_org5.sh
$ vi prepare_sdk_peer0_org6.sh
```

```
$ vi prepare_sdk_peer0_org7.sh
$ vi prepare_sdk_peer0_org8.sh

$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#), [User1@org2.example.com](#), ..., and [User1@org5.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ python prepare_wallet_user1_org3.py
$ python prepare_wallet_user1_org4.py
$ python prepare_wallet_user1_org5.py
$ python prepare_wallet_user1_org6.py
$ python prepare_wallet_user1_org7.py
$ python prepare_wallet_user1_org8.py

$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
root@i27xviqssiy2jdsfcldbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507

// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0180799798489
Mean_Delay_Orderer:
0.00146230137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098003
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer' | awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfcdpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```



```
root@i27xviqssiy2jdsfcldbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// Get block size
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

5. Dynamic Adj on Single Orderer (Exp 5: 1 CLI + 2 Peers + 1 Orderer)

5.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i2/xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderer1 sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i2/xviqssiy2jdsfcl0pc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ...

```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
4a207f175833   hyperledger/fabric-orderer:2.2.0   "orderer"               8 seconds ago Up 8 seconds   0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
38fb1129b2ae   hyperledger/fabric-peer:2.2.0      "peer node start"       2 seconds ago Up 1 second     0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer2.sh
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
e88f655db1d9   hyperledger/fabric-peer:2.2.0      "peer node start"       2 seconds ago Up 1 second     7051/tcp, 0.0.0.0:8051->8051/tcp   peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lp2:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdjhqk0lp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
30e6fbad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"             2 seconds ago Up 1 second                    cli
```

5.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`  
# cd scripts  
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES  
38e67bad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago Up 1 second     
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker exec -it cli bash  
bash-5.0# cd scripts  
bash-5.0# ./step1.sh  
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)  
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized  
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized  
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized  
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized  
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized  
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized  
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received blocks: 0  
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:38.953 UTC [cli.common] readBlock -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
11ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"nGmycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at  
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f50ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f0eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffbf97b289c38f53946f397b242b6d3ff591f2ba4f48aed23911f3] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
200tps  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
bash-5.0#
```

5.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read pem.py
$ vi prepare tls peer0 org1.sh
$ vi prepare tls peer0 org2.sh
$ ./all.sh

2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare wallet user1 org1.py
$ vi prepare wallet user1 org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare sdk peer0 org1.sh
$ vi prepare sdk peer0 org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code

https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare wallet user1 org1.py
```

```
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@i27xviqssiy2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// For example, we can get block size
# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```


5.4. Dynamic Adjust Block Parameters on Single OSN

Step 1 (Run): Orderer fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
root@i27xvqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-09 02:53:08.455 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 02:53:08.457 UTC [cli.common] readBlock -> INFO 002 Received block: 13
2026-05-09 02:53:08.457 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 2
2026-05-09 02:53:08.458 UTC [cli.common] readBlock -> INFO 004 Received block: 2
bash-5.0# ls
channel-artifacts  config_block.pb  crypto  scripts
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

```
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# ls
channel-artifacts  config.json  config_block.pb  crypto  scripts
```

We can check that the initial BatchSize is 60,

```
{
  "values": {
    "BatchSize": {
      "mod_policy": "Admins",
      "value": {
        "absolute_max_bytes": 103809024,
        "max_message_count": 60,
        "preferred_max_bytes": 524288
      }
    },
    "version": "0"
  }
}
```

Step 3 (Run): We update the BatchSize value from 60 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```

```
bash-5.0# BS="channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
60
```

```
# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json
```

```
bash-5.0# jq "$BS = 80" config.json > modified_config.json
bash-5.0# jq "$BS" modified_config.json
80
```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# ls
channel-artifacts  config_block.pb  modified_config.pb
config.json        crypto           scripts
config.pb          modified_config.json
bash-5.0#
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated
modified_config.pb --output final_update.pb
```

```
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# ls
channel-artifacts  config.pb        crypto           modified_config.json  scripts
config.json        config_block.pb  final_update.pb  modified_config.pb
```

Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >
final_update.json
# echo '{"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",
\"type\":2}},\"data\":{\"config_update\":\"$(cat final_update.json)\"}}' | jq . >
header_in_envelope.json
# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output
final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo '{"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":\"$(cat final_update.json)\"}}' | jq
> header_in_envelope.json
bash-5.0# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# ls
channel-artifacts  config_block.pb  final_update.pb  modified_config.json
config.json        crypto           final_update_in_envelope.pb  modified_config.pb
config.pb          final_update.json  header_in_envelope.json      scripts
bash-5.0#
```

Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org1.example.com/users/Admin@org1.example.com/msp
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org2.example.com/users/Admin@org2.example.com/msp
CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 03:09:46.448 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 03:10:01.297 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0#
```

Step 7 (Run): Orderer update the channel as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem
2026-05-09 03:11:23.950 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 03:11:23.961 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
```

```

CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json

```

```

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-09 03:13:24.871 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 03:13:24.873 UTC [cli.common] readBlock -> INFO 002 Received block: 14
2026-05-09 03:13:24.873 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 14
2026-05-09 03:13:24.874 UTC [cli.common] readBlock -> INFO 004 Received block: 14
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# jq "$BS" config.json
80
bash-5.0# █

```

Step 9 (Run): Nodes run the clean procedure,

```

$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh

```

6. Dynamic Adj on Multi Orderers (Exp 6: 1 CLI + 2 Peers + 5 Orderers)

6.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" e
ction ticks:10 heartbeat tick:1 max inflight blocks:5 snapshot_interval_size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first#
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk 100% 241 0.2KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
orderer.example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
server.key 100% 241 0.2KB/s 00:0
server.crt 100% 916 0.9KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
client.crt 100% 814 0.8KB/s 00:0
client.key 100% 241 0.2KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpC2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

```
$ ./orderer2.sh
```

```
root@iZ7xviqssiy2jdsfcldpbwZ:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done
```

```
$ ./orderer3.sh
```

```
root@iZ7xviqssiy2jdsfcldpbZ:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done
```

\$ **./orderer4.sh**

```
root@i27xviqssiy2jdsfclpc3Z:~/fabric-samples/demo-first# ./orderer4.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer4.example.com" with default driver
Creating orderer4.example.com ... done
```

\$ **./orderer5.sh**

```
root@i27xviqssiy2jdsfclpc0Z:~/fabric-samples/demo-first# ./orderer5.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer5.example.com" with default driver
Creating orderer5.example.com ... done
```

\$ docker ps

Step 6 (Run): Peer1 sets up the Docker container,

\$ cd \$HOME/fabric-samples/demo-first

\$ **./peer1.sh**

```
root@i27xviqssiy2jdsfclpc0Z:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

\$ docker ps

Step 7 (Run): Peer2 sets up the Docker container,

\$ cd \$HOME/fabric-samples/demo-first

\$ **./peer2.sh**

```
root@i27xviqssiy2jdsfclpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

\$ docker ps

Step 8 (Run): CLI sets up the Docker container,

\$ cd \$HOME/fabric-samples/demo-first

\$ **./cli.sh**

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

\$ docker ps

6.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash
```

// Environment Configuration for `peer0.org1.example.com:7051`

cd scripts

./step1.sh // All peers join the channel

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto          scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh      step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./step2.sh // All peers join the chaincode

6.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read\_pem.py
$ vi prepare\_tls\_peer0\_org1.sh
$ vi prepare\_tls\_peer0\_org2.sh
$ ./all.sh
```

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare\_wallet\_user1\_org1.py
$ vi prepare\_wallet\_user1\_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
```



```
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare\_sdk\_peer0\_org1.sh
$ vi prepare\_sdk\_peer0\_org2.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare\_wallet\_user1\_org1.py
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
```

```
$ ls result -l
```

```
root@i27xviqssly2jdjshk80lp2:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssly2jdjshk80lp2:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
```

```
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssly2jdjshk80lp2:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer' | awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfcdpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfcdpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// Get block size
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

6.4. Dynamic Adjust Block Parameters on Multiple OSNs

Step 1 (Run): Orderer 1 fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
root@i77xviqssiy2jdjhqk80lp2:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-09 08:42:27.470 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 08:42:27.473 UTC [cli.common] readBlock -> INFO 002 Received block: 15
2026-05-09 08:42:27.473 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 2
2026-05-09 08:42:27.474 UTC [cli.common] readBlock -> INFO 004 Received block: 2
bash-5.0#
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

Step 3 (Run): We update the BatchSize value from 30 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```

```

    },
    "HashingAlgorithm": {
      "mod_policy": "Admins",
      "value": {
        "name": "SHA256"
      },
      "version": "0"
    },
    "OrdererAddresses": {
      "mod_policy": "/Channel/Orderer/Admins",
      "value": {
        "addresses": [
          "orderer.example.com:7050",
          "orderer2.example.com:7050",
          "orderer3.example.com:7050",
          "orderer4.example.com:7050",
          "orderer5.example.com:7050"
        ]
      },
      "version": "0"
    },
    "version": "0"
  },
  "sequence": "3"
}
bash-5.0# BS="channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
30
bash-5.0#

```

```

# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json

```

```

bash-5.0# BS="channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
30
bash-5.0# jq "$BS = 80" config.json > modified_config.json
bash-5.0# jq "$BS" modified_config.json
80
bash-5.0#

```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```

# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb

```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```

# configtxlator compute_update --channel_id mychannel --original config.pb --updated
modified_config.pb --output final_update.pb

```

Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```

# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >
final_update.json
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",
\"type\":2}},\"data\":{\"config_update\":\"$(cat final_update.json)}}}" | jq . >
header_in_envelope.json
# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output
final_update_in_envelope.pb

```

```

bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}}, "data":{"config_update":{"$(cat final_update.json)}}}}' | jq
> header_in_envelope.json
bash-5.0# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0#

```

Step 7 (Run): All endorsing peers sign this update proto, as follows,

```

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org1.example.com/users/Admin@org1.example.com/msp
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

```

```

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org2.example.com/users/Admin@org2.example.com/msp
CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

```

```

bash-5.0# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEER
ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizatio
n/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 08:47:17.445 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp CORE_PEER
ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizatio
n/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 08:47:27.614 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0#

```

Step 7 (Run): Orderer 1 updates the channel as follows,

```

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/order
rerOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/or
dererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o
orderer.example.com:7050 --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/
orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```

```

2026-05-09 08:47:27.614 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=
orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/e
xample.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7
050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-cert.pem
2026-05-09 08:47:51.077 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 08:47:51.089 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#

```

Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-09 08:48:16.878 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 08:48:16.880 UTC [cli.common] readBlock -> INFO 002 Received block: 16
2026-05-09 08:48:16.880 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 16
2026-05-09 08:48:16.881 UTC [cli.common] readBlock -> INFO 004 Received block: 16
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# jq "$BS" config.json
80
bash-5.0#
```

Step 9 (Run): Nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

7. Dynamic Adj on Single Peer (Exp 7: 1 CLI + 1 Peer + 1 Orderer)

7.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock -systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.972 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderer1 sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ...

```

```

$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4a207f175833	hyperledger/fabric-orderer:2.2.0	"orderer"	8 seconds ago	Up 8 seconds	0.0.0.0:7050->7050/tcp	orderer.example.com

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ...
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                    NAMES
38fb1129b2ae   hyperledger/fabric-peer:2.2.0   "peer node start"       2 seconds ago   Up 1 second   0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): -,

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ...
```

```
$ docker ps
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                    NAMES
38e6fbad4c51   hyperledger/fabric-tools:2.2.0   "/bin/bash"             2 seconds ago   Up 1 second                    cli
```

7.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`
# cd scripts
# ./step1.sh // All peers join the channel
```

```
mychannel.block step1.sh step2.sh
bash-5.0# ./step1.sh
2026-05-10 02:37:25.066 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:37:25.081 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-05-10 02:37:25.083 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-05-10 02:37:25.283 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.285 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-05-10 02:37:25.486 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.487 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-05-10 02:37:25.688 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.690 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-05-10 02:37:25.890 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.892 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-05-10 02:37:26.092 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:26.093 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-05-10 02:37:26.295 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-05-10 02:37:26.327 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:37:26.354 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-05-10 02:37:26.380 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:37:26.390 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./step2.sh // All peers join the chaincode

```
go: downloading github.com/xelipuvv/gojsonreference v0.0.0-20180127040603-bd5ef7bd5415
go: downloading github.com/go-openapi/swag v0.19.5
go: downloading github.com/go-openapi/jsonpointer v0.19.3
go: downloading github.com/xelipuvv/gojsonpointer v0.0.0-20180127040702-4e3ac2762d5f
go: downloading github.com/mitru/easyjson v0.0.0-20190626092158-b2ccc519800e
go: downloading github.com/go-openapi/jsonreference v0.19.2
go: downloading gopkg.in/yaml.v2 v2.2.8
go: downloading google.golang.org/genproto v0.0.0-20180831171423-11092d34479b
go: downloading golang.org/x/net v0.0.0-20190827160401-ba9fcec4b297
go: downloading golang.org/x/sys v0.0.0-20190710143415-6ec70d6a5542
go: downloading github.com/PuerkitoBio/purell v1.1.1
go: downloading github.com/PuerkitoBio/urlesc v0.0.0-20170810143723-de5bf2ad4578
go: downloading golang.org/x/text v0.3.2
2026-05-10 02:38:25.043 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfl
dd75670011ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-05-10 02:38:25.043 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3f
8491cdaab7d56219165c0b45
2026-05-10 02:38:25.111 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-05-10 02:38:26.132 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [30bb212defe8d4d076eaba0d797ece8a84471f5ce9c8d167acba96080849715c3] committed with status (VALID) .
{
  "approvals": {
    "Org1MSP": true
  }
}
2026-05-10 02:38:27.287 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [404ce595265b340173ee48b2d6ba3aa468f437f782e964586c8d104754ffbe38d] committed with status (VALID) .
peer0.org1.example.com:7051
2026-05-10 02:38:28.543 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1b603633fccf3d5b09308053d7cd1cf540cd7e2c7c7a394c339def5beab759bf] committed with status (VALID) .
peer0.org1.example.com:7051
2026-05-10 02:38:28.543 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-05-10 02:38:29.590 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [25c5d6a910d52fe7648f638c52780ba45e2ebcac3c7d6abcbd6ca6af8af2341e] committed with status (VALID) .
peer0.org1.example.com:7051
2026-05-10 02:38:29.590 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
bash-5.0#
```

7.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read\_pem.py
$ vi prepare\_tls\_peer0\_org1.sh
$ vi prepare\_tls\_peer0\_org2.sh
$ ./all.sh
```

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare\_wallet\_user1\_org1.py
$ vi prepare\_wallet\_user1\_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare\_sdk\_peer0\_org1.sh
$ vi prepare\_sdk\_peer0\_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code

https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare\_wallet\_user1\_org1.py
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
```

```

Delay_Endorsing: 0.018999814987182617
Get_Endorsed_Proposal: 1778380771.862
Delay_Endorsing: 0.020999908447265625
Get_Ordered_Transaction: 1778380771.863
Delay_Ordering: 0.0009999275207519531
Get_Endorsed_Proposal: 1778380771.865
Delay_Endorsing: 0.023999920428100586
Get_Endorsed_Proposal: 1778380771.867
Delay_Endorsing: 0.026000022888183594
Get_Ordered_Transaction: 1778380771.867
Delay_Ordering: 0
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Send_Endorsing_Proposal: 1778380772.206
Send_Endorsing_Proposal: 1778380772.208
Send_Endorsing_Proposal: 1778380772.209
Send_Endorsing_Proposal: 1778380772.211
Send_Endorsing_Proposal: 1778380772.212
Send_Endorsing_Proposal: 1778380772.214
Send_Endorsing_Proposal: 1778380772.215
Send_Endorsing_Proposal: 1778380772.217

```

```

// $ node query.js
// $ node invoke.js 30 500 20 1

```

Step 5 (Run): Analyze the log,

```

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs

```

```

root@i7xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178

```

```

$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase

```

```

root@i7xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0180799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002090003

```

```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
// $ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer' | awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks

```

```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
// $ docker logs containername >& peerlog

```

```
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// For example, we can get block size
# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

7.4. Dynamic Adjust Block Parameters on Single Peer

Step 1 (Run): Orderer fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-10 02:40:38.286 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:40:38.288 UTC [cli.common] readBlock -> INFO 002 Received block: 9
2026-05-10 02:40:38.288 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 1
2026-05-10 02:40:38.289 UTC [cli.common] readBlock -> INFO 004 Received block: 1
bash-5.0# █
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |  
jq .data.data[0].payload.data.config > config.json
```

```
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json  
bash-5.0# ls  
channel-artifacts  config.json  config_block.pb  crypto  scripts
```

We can check that the initial BatchSize is 60,

```
"Writers": {  
  "mod_policy": "Admins",  
  "policy": {  
    "type": 3,  
    "value": {  
      "rule": "ANY",  
      "sub_policy": "Writers"  
    }  
  },  
  "version": "0"  
},  
"values": {  
  "BatchSize": {  
    "mod_policy": "Admins",  
    "value": {  
      "absolute_max_bytes": 103809024,  
      "max_message_count": 60,  
      "preferred_max_bytes": 524288  
    }  
  },  
  "version": "0"  
},  
"BatchTimeout": {  
  "mod_policy": "Admins",  
  "value": {
```

Step 3 (Run): We update the BatchSize value from 60 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"  
# jq "$BS" config.json
```

```
bash-5.0# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"  
bash-5.0# jq "$BS" config.json  
60
```

```
# jq "$BS = 80" config.json > modified_config.json  
# jq "$BS" modified_config.json
```

```
bash-5.0# jq "$BS = 80" config.json > modified_config.json  
bash-5.0# jq "$BS" modified_config.json  
80
```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb  
# configtxlator proto_encode --input modified_config.json --type common.Config --output  
modified_config.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb  
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb  
bash-5.0# ls  
channel-artifacts  config_block.pb  modified_config.pb  
config.json        crypto           scripts  
config.pb          modified_config.json  
bash-5.0#
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated  
modified_config.pb --output final_update.pb
```

```
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb  
bash-5.0# ls  
channel-artifacts    config.pb             crypto               modified_config.json  scripts  
config.json          config_block.pb      final_update.pb     modified_config.pb  
bash-5.0#
```

Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >  
final_update.json  
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",  
\"type\":\"2\"}},\"data\":{\"config_update\":{\"$(cat final_update.json)}}}}\" | jq . >  
header_in_envelope.json  
# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output  
final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json  
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":\"2\"}},\"data\":{\"config_update\":{\"$(cat final_update.json)}}}}\" | jq  
> header_in_envelope.json  
bash-5.0# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output final_update_in_envelope.pb  
bash-5.0# ls  
channel-artifacts    config_block.pb      final_update.pb      modified_config.json  
config.json          crypto              final_update_in_envelope.pb  modified_config.pb  
config.pb            final_update.json   header_in_envelope.json  scripts  
bash-5.0#
```

Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
#  
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer  
Organizations/org1.example.com/users/Admin@org1.example.com/msp  
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"  
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe  
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel  
signconfigtx -f final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb  
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb  
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb  
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json  
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":\"2\"}},\"data\":{\"config_update\":{\"$(cat final_update.json)}}}}\" | jq  
> header_in_envelope.json  
bash-5.0# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output final_update_in_envelope.pb  
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEE  
ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations  
/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb  
2026-05-10 02:42:58.035 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
bash-5.0#
```

Step 7 (Run): Orderer update the channel as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-10 02:43:28.440 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:43:28.449 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json
```

```
2026-05-10 02:43:28.449 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-10 02:43:55.776 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:43:55.778 UTC [cli.common] readBlock -> INFO 002 Received block: 10
2026-05-10 02:43:55.778 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 10
2026-05-10 02:43:55.779 UTC [cli.common] readBlock -> INFO 004 Received block: 10
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# jq "$BS" config.json
80
bash-5.0#
```

Step 9 (Run): Nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```


8. Dynamic Adj on Multi Peers (Exp 8: 1 CLI + 2 Peers + 1 Orderer)

8.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i2/xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderer1 sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i2/xviqssiy2jdsfcl0pc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ...

```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
4a207f175833   hyperledger/fabric-orderer:2.2.0   "orderer"               8 seconds ago Up 8 seconds   0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
38fb1129b2ae   hyperledger/fabric-peer:2.2.0       "peer node start"       2 seconds ago Up 1 second     0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer2.sh
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
e88f655db1d9   hyperledger/fabric-peer:2.2.0       "peer node start"       2 seconds ago Up 1 second     7051/tcp, 0.0.0.0:8051->8051/tcp   peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lp2:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

```
root@i27xviqssiy2jdjhqk0lp2:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
30e6fbad4c51   hyperledger/fabric-tools:2.2.0      "/bin/bash"             2 seconds ago Up 1 second                    cli
```

8.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`  
# cd scripts  
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# docker ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS          NAMES  
38e67bad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago   Up 1 second      
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# docker exec -it cli bash  
bash-5.0# cd scripts  
bash-5.0# ./step1.sh  
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)  
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized  
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized  
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized  
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized  
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized  
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized  
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received blocks: 0  
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
11ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at  
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:22.230 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f50ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f0eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffbf97b289c38f53946f397b242b6d3ff591f2ba4f48aed23911f3] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
200tps  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
bash-5.0#
```

8.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read pem.py
$ vi prepare tls peer0 org1.sh
$ vi prepare tls peer0 org2.sh
$ ./all.sh

2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare wallet user1 org1.py
$ vi prepare wallet user1 org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare sdk peer0 org1.sh
$ vi prepare sdk peer0 org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code

https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare wallet user1 org1.py
```

```
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@i27xviqssiy2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// For example, we can get block size
# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

8.4. Dynamic Adjust Block Parameters on Multiple Peers

Step 1 (Run): Orderer fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
root@i27xvqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-09 02:53:08.455 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 02:53:08.457 UTC [cli.common] readBlock -> INFO 002 Received block: 13
2026-05-09 02:53:08.457 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 2
2026-05-09 02:53:08.458 UTC [cli.common] readBlock -> INFO 004 Received block: 2
bash-5.0# ls
channel-artifacts  config_block.pb  crypto  scripts
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

```
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# ls
channel-artifacts  config.json  config_block.pb  crypto  scripts
```

We can check that the initial BatchSize is 60,

```
{
  "values": {
    "BatchSize": {
      "mod_policy": "Admins",
      "value": {
        "absolute_max_bytes": 103809024,
        "max_message_count": 60,
        "preferred_max_bytes": 524288
      }
    },
    "version": "0"
  }
}
```

Step 3 (Run): We update the BatchSize value from 60 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```



```
bash-5.0# BS="channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
60
```

```
# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json
```

```
bash-5.0# jq "$BS = 80" config.json > modified_config.json
bash-5.0# jq "$BS" modified_config.json
80
```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# ls
channel-artifacts  config_block.pb  modified_config.pb
config.json        crypto           scripts
config.pb          modified_config.json
bash-5.0#
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated
modified_config.pb --output final_update.pb
```

```
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# ls
channel-artifacts  config.pb        crypto           modified_config.json  scripts
config.json        config_block.pb  final_update.pb  modified_config.pb
```

Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >
final_update.json
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",
\"type\":2}},\"data\":{\"config_update\":\"$(cat final_update.json)\"}}}" | jq . >
header_in_envelope.json
# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output
final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":\"$(cat final_update.json)\"}}}" | jq
> header_in_envelope.json
bash-5.0# configtxlator proto_encode --input header_in_envelope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# ls
channel-artifacts  config_block.pb  final_update.pb  modified_config.json
config.json        crypto           final_update_in_envelope.pb  modified_config.pb
config.pb          final_update.json  header_in_envelope.json      scripts
bash-5.0#
```

Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org1.example.com/users/Admin@org1.example.com/msp
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org2.example.com/users/Admin@org2.example.com/msp
CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 03:09:46.448 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 03:10:01.297 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0#
```

Step 7 (Run): Orderer update the channel as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsca.example.com-cert.pem

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsca.example.com-cert.pem
2026-05-09 03:11:23.950 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 03:11:23.961 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
```

```

CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json

```

```

bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel -tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-09 03:13:24.871 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 03:13:24.873 UTC [cli.common] readBlock -> INFO 002 Received block: 14
2026-05-09 03:13:24.873 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 14
2026-05-09 03:13:24.874 UTC [cli.common] readBlock -> INFO 004 Received block: 14
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# jq "$BS" config.json
00
bash-5.0# █

```

Step 9 (Run): Nodes run the clean procedure,

```

$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh

```

9. Different Transaction Payoff Sizes (Exp 9: 1 CLI + 2 Peers + 1 Orderer)

9.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i2/xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat tick:1 max inflight blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderer1 sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i2/xviqssiy2jdsfcldp22:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ...

```

```

$ docker ps

```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4a207f175833	hyperledger/fabric-orderer:2.2.0	"orderer"	8 seconds ago	Up 8 seconds	0.0.0.0:7050->7050/tcp	orderer.example.com

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
38fb1129b2ae	hyperledger/fabric-peer:2.2.0	"peer node start"	2 seconds ago	Up 1 second	0.0.0.0:7051->7051/tcp	peer0.org1.example.com

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh
```

```
root@i27xviqssiy2jdsfcldp2:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e88f655db1d9	hyperledger/fabric-peer:2.2.0	"peer node start"	2 seconds ago	Up 1 second	7051/tcp, 0.0.0.0:8051->8051/tcp	peer0.org2.example.com

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lp2:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
30e6fbad4c51	hyperledger/fabric-tools:2.2.0	"/bin/bash"	2 seconds ago	Up 1 second		cli

9.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`  
# cd scripts  
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# docker ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES  
38e67bad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago Up 1 second     
root@i27xviqssiy2jdjhqk0lpz:~/fabric-samples/demo-first# docker exec -it cli bash  
bash-5.0# cd scripts  
bash-5.0# ./step1.sh  
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)  
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized  
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized  
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized  
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized  
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized  
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized  
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received blocks: 0  
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:38.953 UTC [cli.common] readBlock -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
11ac10e3978491cdaab7d56219165c0b45022006mycc_1* >  
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"nGmycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cdaab7d56219165c0b45022006mycc_1* >  
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at  
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f50ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f0eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffbf97b289c38f53946f397b242b6d3ff591f2ba4f48aed23911f3] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
200tps  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
bash-5.0#
```

9.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read pem.py
$ vi prepare tls peer0 org1.sh
$ vi prepare tls peer0 org2.sh
$ ./all.sh

2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare wallet user1 org1.py
$ vi prepare wallet user1 org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare sdk peer0 org1.sh
$ vi prepare sdk peer0 org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code

https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare wallet user1 org1.py
```



```
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@i27xviqssiy2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
```

```
// $ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer' | awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfclpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
// $ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py
```

```
root@i27xviqssiy2jdsfclpc2Z:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// For example, we can get block size
# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

9.4. Different Transaction Payoff Sizes

Step 1 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
```

[illegible]

```
$ cd $HOME/fabric-samples/demo-first/workload
```

[illegible]

Step 3 (Run): CLI invokes and queries transactions via Node.js,

```
$ ./ssh.sh > result/benchlog // Transaction payoff size: 100 byte
$ ls result -l
```

10. Different Chaincodes (Exp 10: 1 CLI + 2 Peers + 1 Orderer)

10.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i2/xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat tick:1 max inflight blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderer1 sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i2/xviqssiy2jdsfcl0pc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ...

```

```

$ docker ps

```

```
root@i27xviqssiy2jdsfcldp2c2:~/fabric-samples/demo-first# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4a207f175833	hyperledger/fabric-orderer:2.2.0	"orderer"	8 seconds ago	Up 8 seconds	0.0.0.0:7050->7050/tcp	orderer.example.com

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldpbv2:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
38fb1129b2ae	hyperledger/fabric-peer:2.2.0	"peer node start"	2 seconds ago	Up 1 second	0.0.0.0:7051->7051/tcp	peer0.org1.example.com

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh
```

```
root@i27xviqssiy2jdsfcldpcl2:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e88f655db1d9	hyperledger/fabric-peer:2.2.0	"peer node start"	2 seconds ago	Up 1 second	7051/tcp, 0.0.0.0:8051->8051/tcp	peer0.org2.example.com

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lp2:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
30e6fbad4c51	hyperledger/fabric-tools:2.2.0	"/bin/bash"	2 seconds ago	Up 1 second		cli

10.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`  
# cd scripts  
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# docker ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES  
38e67bad4c51   hyperledger/fabric-tools:2.2.0     "/bin/bash"            2 seconds ago Up 1 second     
root@i27xviqssiy2jdjhqk0olpz:~/fabric-samples/demo-first# docker exec -it cli bash  
bash-5.0# cd scripts  
bash-5.0# ./step1.sh  
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)  
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized  
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized  
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized  
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized  
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized  
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)  
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized  
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received blocks: 0  
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel  
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized  
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update  
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
11ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"nGmycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cdaab7d56219165c0b45022006mycc_1" >  
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cd:  
b7d56219165c0b45  
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at  
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050  
2026-04-14 08:27:22.230 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f50ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f0eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdccfd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffbf97b289c38f53946f397b242b6d3ff591f2ba4f48aed239311f3] committed with status (VALID) at peer0  
rg1.example.com:7051  
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
200tps  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0  
rg2.example.com:8051  
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200  
bash-5.0#
```


10.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read pem.py
$ vi prepare tls peer0 org1.sh
$ vi prepare tls peer0 org2.sh
$ ./all.sh

2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare wallet user1 org1.py
$ vi prepare wallet user1 org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare sdk peer0 org1.sh
$ vi prepare sdk peer0 org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code

https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare wallet user1 org1.py
```

```
$ python prepare\_wallet\_user1\_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@i27xviqssiy2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

```
// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
```

```
// $ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfclpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
// $ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfclpc2Z:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// For example, we can get block size
# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

10.4. Different Chaincodes

Step 1 (Run): CLI uses a different chaincode as follows,

```
$ cd $HOME/fabric-samples/chaincode/abstore/go
$ vi abstore.go
```

```
$ ls
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# ls
abstore.go go.mod go.sum mycc.tar.gz vendor
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go#
```

Step 2 (Run): CLI compiles the chaincode in Go, and ensures that the chaincode in Go has no grammar errors, as follows,

```
$ go mod vendor
```

```
$ go build
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# go mod vendor
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# go build
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# ls
abstore.go go go.mod go.sum mycc.tar.gz vendor
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go#
```

Step 3 (Run): CLI modifies the step2.sh, and ensures that the new chaincode can be initialized, as follows,

```
$ cd $HOME/fabric-samples/demo-first/scripts
```

```
$ vi step2.sh
```

```
# Peer0.org1.example.com init
peer chaincode invoke -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C $CHANNEL_NAME -n mycc --peerAddresses peer0.org1.example.com:7051 --tlsRootCertFiles /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt -c '{"Args":["Init","a","100","b","50"]}' --waitForEvent

# Peer0.org1.example.com invoke
peer chaincode invoke -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C $CHANNEL_NAME -n mycc --peerAddresses peer0.org1.example.com:7051 --tlsRootCertFiles /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt -c '{"Args":["Invoke","a","b","200string"]}' --waitForEvent

peer chaincode query -C $CHANNEL_NAME -n mycc -c '{"Args":["Query","a"]}'
peer chaincode query -C $CHANNEL_NAME -n mycc -c '{"Args":["Query","b"]}'
```

Step 4 (Run): CLI modifies all peers' workload generator, as follows,

```
$ cd $HOME/fabric-samples/demo-first/workload
```

```
$ vi prepare_sdk_peer0_org1.sh
```

```
// -----
// After load, submit a transaction
// -----
function submit_a_tx_after_the_setting(){
    var t1 = Date.now()/1000;
    //console.log('Send Endorsing Proposal: %j', t1);
    //contract.submitTransaction('invoke', 'a', '330dota');
    contract.submitTransaction('invoke', 'a', 'b', makeid(process.argv[5]));
    //var t2 = Date.now()/1000;
    //console.log('Get Endorsed Proposal: %j', t2);
    //console.log("This transaction Finished on: ", new Date, "\n")
}
```

103,1 48%

\$ vi prepare_sdk_peer0_org2.sh

```
        result += characters.charAt(Math.floor(Math.random() * charactersLength));
    }
    return result;
}
// -----
// After load, submit a transaction
// -----
function submit_a_tx_after_the_setting(){
    var t1 = Date.now()/1000;
    //console.log('Send Endorsing Proposal: %j', t1);
    //contract.submitTransaction('invoke', 'a', '330dota');
    contract.submitTransaction('invoke', 'a', 'b', makeid(process.argv[5]));
    //var t2 = Date.now()/1000;
    //console.log('Get Endorsed Proposal: %j', t2);
    //console.log("This transaction Finished on: ", new Date, "\n")
}
function workload(transactions){
    try{
        for(var i = 0; i < transactions; i++){
```

102,1 50%

Step 5 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
2026-05-12 09:09:47.439 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-05-12 09:09:47.439 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-05-12 09:09:47.440 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-05-12 09:09:47.454 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-05-12 09:09:47.471 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-05-12 09:09:47.471 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-05-12 09:09:47.472 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-05-12 09:09:47.486 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-05-12 09:09:47.502 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-05-12 09:09:47.502 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-05-12 09:09:47.503 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first#
```

Step 6 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./scp.sh
```

Step 7 (Run): Set up all components as follows,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./orderer1.sh  
  
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh  
Creating volume "net_orderer.example.com" with default driver  
Creating orderer.example.com ... done  
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first#
```

```
$ cd $HOME/fabric-samples/demo-first  
$ ./peer1.sh  
  
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh  
Creating volume "net_peer0.org1.example.com" with default driver  
Creating peer0.org1.example.com ... done  
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first#
```

```
$ cd $HOME/fabric-samples/demo-first  
$ ./peer2.sh  
  
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh  
Creating volume "net_peer0.org2.example.com" with default driver  
Creating peer0.org2.example.com ... done  
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first#
```

```
$ cd $HOME/fabric-samples/demo-first  
$ ./cli.sh  
  
root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./cli.sh  
Creating cli ... done  
root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first#
```

Step 8 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash  
  
// Environment Configuration for `peer0.org1.example.com:7051`
```

```
# cd scripts
# ./step1.sh // All peers join the channel
```

```
ed
2026-05-12 01:16:46.280 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-12 01:16:46.282 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-05-12 01:16:46.485 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-05-12 01:16:46.515 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-12 01:16:46.537 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-05-12 01:16:46.566 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-12 01:16:46.587 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-05-12 01:16:46.615 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-12 01:16:46.626 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-05-12 01:16:46.653 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-12 01:16:46.665 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

```
2026-05-12 01:17:23.109 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:408cde5495ec4730e6b47f1b0732d0972d64bbc14955409866c36123019985dc
2026-05-12 01:17:23.179 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-05-12 01:17:24.203 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a0fa36507f5b99c6f2fb0811bda8bc2b5385285f12a50c0656c8ad6e232862da] committed with status (VALID) at
2026-05-12 01:17:24.239 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-05-12 01:17:25.259 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [9d3b0183ef40b8c0b8914bd5bd8fa32f6cfd25fcafa9bcee8e31da2227266390] committed with status (VALID) at
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
2026-05-12 01:17:26.541 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [76fdac61e10ef75f8b48f3f2ed6559ad00061488ded2a988e714eb6e709363fe] committed with status (VALID) at peer0.org2.example.com:8051
2026-05-12 01:17:26.543 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [76fdac61e10ef75f8b48f3f2ed6559ad00061488ded2a988e714eb6e709363fe] committed with status (VALID) at peer0.org1.example.com:7051
2026-05-12 01:17:27.704 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [5b9c53e1f4f0ee94b023948fcf5ca14c99639cbac7602e95d2f7a1b677d9ebc5] committed with status (VALID) at peer0.org1.example.com:7051
2026-05-12 01:17:27.704 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-05-12 01:17:28.753 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [c9385d3760dfab3d970038c64c90f2875e35810c2b23ffda8f49005a07fb9273] committed with status (VALID) at peer0.org1.example.com:7051
2026-05-12 01:17:28.753 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200string
200string
bash-5.0#
```

Step 9 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ ./all.sh
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ ./all.sh
```

Step 10 (Run): CLI invokes and queries transactions via Node.js,

[illegible]

11. Run Multi-Nodes Fabric (Exp 11: 1 CLI + 8 Peers + 3 Orderers)

11.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli

172.31.210.70 peer0.org1.example.com # endorsing peer
172.31.210.71 peer0.org2.example.com # endorsing peer
172.31.210.77 peer0.org3.example.com # endorsing peer
172.31.210.78 peer0.org4.example.com # endorsing peer
172.31.210.79 peer0.org5.example.com # endorsing peer
172.31.210.82 peer0.org6.example.com # endorsing peer
172.31.210.83 peer0.org7.example.com # endorsing peer
172.31.210.84 peer0.org8.example.com # non-endorsing peer

172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
$ vi peer0-org3.yaml
$ vi peer0-org4.yaml
$ vi peer0-org5.yaml
$ vi peer0-org6.yaml
$ vi peer0-org7.yaml
$ vi peer0-org8.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh

root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" e
ction tick:10 heartbeat tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
root@iZ7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first#
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

priv_sk 100% 241 0.2KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
orderer.example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
server.key 100% 241 0.2KB/s 00:0
server.crt 100% 916 0.9KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
client.crt 100% 814 0.8KB/s 00:0
client.key 100% 241 0.2KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

```
$ ./orderer2.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done
```

```
$ ./orderer3.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh
```

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ ./peer3.sh
```

```
$ ./peer4.sh
```

```
$ ./peer5.sh
```

```
$ ./peer6.sh
```

```
$ ./peer7.sh
```

```
$ ./peer8.sh
```

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0olp2:~/fabric-samples/demo-first# ./cli.sh
```

```
Creating cli ... done
```

```
$ docker ps
```

11.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
```

```
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`
```

```
# cd scripts
```

```
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0olp2:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto          scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh      step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT FOUND)
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

11.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
```

```
$ vi read\_pem.py
```

```
$ vi prepare\_tls\_peer0\_org1.sh
```

```
$ vi prepare\_tls\_peer0\_org2.sh
$ vi prepare_tls_peer0_org3.sh
$ vi prepare_tls_peer0_org4.sh
$ vi prepare_tls_peer0_org5.sh
$ vi prepare_tls_peer0_org6.sh
$ vi prepare_tls_peer0_org7.sh
$ vi prepare_tls_peer0_org8.sh
$ ./all.sh
```

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare\_wallet\_user1\_org1.py
$ vi prepare\_wallet\_user1\_org2.py
$ vi prepare_wallet_user1_org3.py
$ vi prepare_wallet_user1_org4.py
$ vi prepare_wallet_user1_org5.py
$ vi prepare_wallet_user1_org6.py
$ vi prepare_wallet_user1_org7.py
$ vi prepare_wallet_user1_org8.py
$ ./all.sh
```

```
$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ mkdir sdk-peer0-org3
$ mkdir sdk-peer0-org4
$ mkdir sdk-peer0-org5
$ mkdir sdk-peer0-org6
$ mkdir sdk-peer0-org7
$ mkdir sdk-peer0-org8
```

```
$ vi invoke.js
$ vi prepare\_sdk\_peer0\_org1.sh
$ vi prepare\_sdk\_peer0\_org2.sh
$ vi prepare_sdk_peer0_org3.sh
$ vi prepare_sdk_peer0_org4.sh
$ vi prepare_sdk_peer0_org5.sh
$ vi prepare_sdk_peer0_org6.sh
$ vi prepare_sdk_peer0_org7.sh
$ vi prepare_sdk_peer0_org8.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare\_wallet\_user1\_org1.py
$ python prepare\_wallet\_user1\_org2.py
$ python prepare_wallet_user1_org3.py
$ python prepare_wallet_user1_org4.py
$ python prepare_wallet_user1_org5.py
$ python prepare_wallet_user1_org6.py
$ python prepare_wallet_user1_org7.py
$ python prepare_wallet_user1_org8.py
$ ./all.sh

```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```

$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
root@i27xviqssiy2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507

// $ node query.js
// $ node invoke.js 30 500 20 1

```

Step 5 (Run): Analyze the log,

```

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput\_p1.py // rate of sending endorsed txs

root@i27xviqssiy2jdhqk0lpz:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178

$ sudo apt-get install python-numpy
$ python readdelay\_p1.py // delay of endorsing phase

```

```

root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137217
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098883

```

```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ docker logs $(docker ps -a | grep 'orderer' | awk '{print $1 }') >& ordererlog
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput\_p2.py // rate of generating blocks

```

```

root@i27xviqssiy2jdsfcdpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]

```

```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py

```

```

root@i27xviqssiy2jdsfcdpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

```

```

// Get block size
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./clean.sh
```


12. Run Multi-Nodes Fabric (Exp 12: 1 CLI + 8 Peers + 9 Orderers)

12.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli

172.31.210.70 peer0.org1.example.com # endorsing peer
172.31.210.71 peer0.org2.example.com # endorsing peer
172.31.210.77 peer0.org3.example.com # endorsing peer
172.31.210.78 peer0.org4.example.com # endorsing peer
172.31.210.79 peer0.org5.example.com # endorsing peer
172.31.210.82 peer0.org6.example.com # endorsing peer
172.31.210.83 peer0.org7.example.com # endorsing peer
172.31.210.84 peer0.org8.example.com # non-endorsing peer

172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
172.31.210.85 orderer6.example.com
172.31.210.86 orderer7.example.com
172.31.210.87 orderer8.example.com
172.31.210.88 orderer9.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
$ vi peer0-org3.yaml
$ vi peer0-org4.yaml
$ vi peer0-org5.yaml
$ vi peer0-org6.yaml
$ vi peer0-org7.yaml
$ vi peer0-org8.yaml

$ vi orderer1.yaml
```

```
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml
$ vi orderer6.yaml
$ vi orderer7.yaml
$ vi orderer8.yaml
$ vi orderer9.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" e
ction tick:10 heartbeat tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

priv_sk	100%	241	0.2KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
orderer.example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
server.key	100%	241	0.2KB/s	00:0
server.crt	100%	916	0.9KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
tlsca.example.com-cert.pem	100%	847	0.8KB/s	00:0
Admin@example.com-cert.pem	100%	769	0.8KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.crt	100%	847	0.8KB/s	00:0
client.crt	100%	814	0.8KB/s	00:0
client.key	100%	241	0.2KB/s	00:0
priv_sk	100%	241	0.2KB/s	00:0
ca.example.com-cert.pem	100%	839	0.8KB/s	00:0

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfclp2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

```
$ ./orderer2.sh
```

```
root@iZ7xviqssiy2jdsfclp2Z:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done
```

```
$ ./orderer3.sh
```

```
root@iZ7xviqssiy2jdsfclp2Z:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done
```

```
$ ./orderer4.sh
```

```
$ ./orderer5.sh
```

```
$ ./orderer6.sh
```

```
$ ./orderer7.sh
```

```
$ ./orderer8.sh
```

```
$ ./orderer9.sh
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
```

```
$ ./peer1.sh
```

```
root@iZ7xviqssiy2jdsfclp2Z:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./peer2.sh
```

```
root@iZ7xviqssiy2jdsfcldpclZ:~/fabric-samples/demo-first# ./peer2.sh  
Creating volume "net_peer0.org2.example.com" with default driver  
Creating peer0.org2.example.com ... done
```

```
$ ./peer3.sh
```

```
$ ./peer4.sh
```

```
$ ./peer5.sh
```

```
$ ./peer6.sh
```

```
$ ./peer7.sh
```

```
$ ./peer8.sh
```

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./cli.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh  
Creating cli ... done
```

```
$ docker ps
```

12.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`  
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`  
# cd scripts  
# ./step1.sh // All peers join the channel
```

```

root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto          scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh      step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#

```

./step2.sh // All peers join the chaincode

12.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```

# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read\_pem.py
$ vi prepare\_tls\_peer0\_org1.sh
$ vi prepare\_tls\_peer0\_org2.sh
$ vi prepare_tls_peer0_org3.sh
$ vi prepare_tls_peer0_org4.sh
$ vi prepare_tls_peer0_org5.sh
$ vi prepare_tls_peer0_org6.sh
$ vi prepare_tls_peer0_org7.sh
$ vi prepare_tls_peer0_org8.sh
$ ./all.sh

2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare\_wallet\_user1\_org1.py
$ vi prepare\_wallet\_user1\_org2.py
$ vi prepare_wallet_user1_org3.py

```

```
$ vi prepare_wallet_user1_org4.py
$ vi prepare_wallet_user1_org5.py
$ vi prepare_wallet_user1_org6.py
$ vi prepare_wallet_user1_org7.py
$ vi prepare_wallet_user1_org8.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ mkdir sdk-peer0-org3
$ mkdir sdk-peer0-org4
$ mkdir sdk-peer0-org5
$ mkdir sdk-peer0-org6
$ mkdir sdk-peer0-org7
$ mkdir sdk-peer0-org8

$ vi invoke.js
$ vi prepare\_sdk\_peer0\_org1.sh
$ vi prepare\_sdk\_peer0\_org2.sh
$ vi prepare_sdk_peer0_org3.sh
$ vi prepare_sdk_peer0_org4.sh
$ vi prepare_sdk_peer0_org5.sh
$ vi prepare_sdk_peer0_org6.sh
$ vi prepare_sdk_peer0_org7.sh
$ vi prepare_sdk_peer0_org8.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of [User1@org1.example.com](#) and [User1@org2.example.com](#),

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare\_wallet\_user1\_org1.py
$ python prepare\_wallet\_user1\_org2.py
$ python prepare_wallet_user1_org3.py
$ python prepare_wallet_user1_org4.py
$ python prepare_wallet_user1_org5.py
$ python prepare_wallet_user1_org6.py
$ python prepare_wallet_user1_org7.py
$ python prepare_wallet_user1_org8.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
```

```
$ ls result -l
```

```
root@iZ7xviqssiy2jdsfclpdpvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfclpdpvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
```

```
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ python readthroughput\_p1.py // rate of sending endorsed txs
```

```
root@iZ7xviqssiy2jdhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

```
$ sudo apt-get install python-numpy
```

```
$ python readdelay\_p1.py // delay of endorsing phase
```

```
root@iZ7xviqssiy2jdhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959802
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002090883
```

```
// Orderer analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ sudo apt-get install python-numpy
```

```
//$ docker logs containername >& ordererlog
```

```
$ docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog
```

```
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
```

```
$ python readthroughput\_p2.py // rate of generating blocks
```

```
root@i27xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]
lastPersistedBlock=[25]
lastPersistedBlock=[26]
lastPersistedBlock=[27]
lastPersistedBlock=[28]
lastPersistedBlock=[29]
lastPersistedBlock=[30]
lastPersistedBlock=[31]
lastPersistedBlock=[32]
```

```
// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ docker logs $(docker ps -a | grep 'peer node start' | awk '{print $1 }') >& peerlog
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput\_p3.py
```

```
root@i27xviqssiy2jdsfcldbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

```
// Get block size
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```


13. Run Multi-Nodes Fabric (Exp 13: 1 CLI + 8 Peers + 15 Orderers)

13.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli

172.31.210.70 peer0.org1.example.com # endorsing peer
172.31.210.71 peer0.org2.example.com # endorsing peer
172.31.210.77 peer0.org3.example.com # endorsing peer
172.31.210.78 peer0.org4.example.com # endorsing peer
172.31.210.79 peer0.org5.example.com # endorsing peer
172.31.210.82 peer0.org6.example.com # endorsing peer
172.31.210.83 peer0.org7.example.com # endorsing peer
172.31.210.84 peer0.org8.example.com # non-endorsing peer

172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
172.31.210.85 orderer6.example.com
172.31.210.86 orderer7.example.com
172.31.210.87 orderer8.example.com
172.31.210.88 orderer9.example.com
172.31.210.89 orderer10.example.com
172.31.210.90 orderer11.example.com
172.31.210.91 orderer12.example.com
172.31.210.92 orderer13.example.com
172.31.210.93 orderer14.example.com
172.31.210.94 orderer15.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
```

```
$ vi peer0-org3.yaml
$ vi peer0-org4.yaml
$ vi peer0-org5.yaml
$ vi peer0-org6.yaml
$ vi peer0-org7.yaml
$ vi peer0-org8.yaml
```

```
$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml
$ vi orderer6.yaml
$ vi orderer7.yaml
$ vi orderer8.yaml
$ vi orderer9.yaml
$ vi orderer10.yaml
$ vi orderer11.yaml
$ vi orderer12.yaml
$ vi orderer13.yaml
$ vi orderer14.yaml
$ vi orderer15.yaml
```

```
$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock --systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```

root@i7xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" e
ction ticks:10 heartbeat ticks:1 max inflight blocks:5 snapshot_interval_size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update

```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```

$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh

```

```

priv_sk 100% 241 0.2KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
orderer.example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
server.key 100% 241 0.2KB/s 00:0
server.crt 100% 916 0.9KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
tlsca.example.com-cert.pem 100% 847 0.8KB/s 00:0
Admin@example.com-cert.pem 100% 769 0.8KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.crt 100% 847 0.8KB/s 00:0
client.crt 100% 814 0.8KB/s 00:0
client.key 100% 241 0.2KB/s 00:0
priv_sk 100% 241 0.2KB/s 00:0
ca.example.com-cert.pem 100% 839 0.8KB/s 00:0

```

Step 5 (Run): Orderers sets up the Docker container,

```

$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

```

```

root@i7xviqssiy2jdsfcldpC2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done

```

```

$ ./orderer2.sh

```

```

root@i7xviqssiy2jdsfcldpbwZ:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done

```

```

$ ./orderer3.sh

```

```

root@i7xviqssiy2jdsfcldpbZ:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done

```

```

$ ./orderer4.sh

```

```

$ ./orderer5.sh

```

```

$ ./orderer6.sh

```

```
$ ./orderer7.sh
$ ./orderer8.sh
$ ./orderer9.sh
$ ./orderer10.sh
$ ./orderer11.sh
$ ./orderer12.sh
$ ./orderer13.sh
$ ./orderer14.sh
$ ./orderer15.sh
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@127xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

Step 7 (Run): Peer2 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh
```

```
root@127xviqssiy2jdsfcldpclZ:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

```
$ ./peer3.sh
$ ./peer4.sh
$ ./peer5.sh
$ ./peer6.sh
$ ./peer7.sh
$ ./peer8.sh
```

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

13.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash
```

```
// Environment Configuration for `peer0.org1.example.com:7051`
# cd scripts
# ./step1.sh // All peers join the channel
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto          scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh      step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &(NOT_FOUND)
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &(SERVICE_UNAVAILABLE)
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

```
# ./step2.sh // All peers join the chaincode
```

13.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
```

```
$ vi read\_pem.py
```

```
$ vi prepare\_tls\_peer0\_org1.sh
```

```
$ vi prepare\_tls\_peer0\_org2.sh
```

```
$ vi prepare_tls_peer0_org3.sh
```

```
$ vi prepare_tls_peer0_org4.sh
```

```
$ vi prepare_tls_peer0_org5.sh
```

```
$ vi prepare_tls_peer0_org6.sh
```

```
$ vi prepare_tls_peer0_org7.sh
```

```
$ vi prepare_tls_peer0_org8.sh
```

```
$ ./all.sh
```

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
```

```
$ vi prepare\_wallet\_user1\_org1.py
```

```
$ vi prepare\_wallet\_user1\_org2.py
```

```
$ vi prepare_wallet_user1_org3.py
```

```
$ vi prepare_wallet_user1_org4.py
```

```
$ vi prepare_wallet_user1_org5.py
```

```
$ vi prepare_wallet_user1_org6.py
```

```
$ vi prepare_wallet_user1_org7.py
```

```
$ vi prepare_wallet_user1_org8.py
```

```
$ ./all.sh
```

```
$ cd $HOME/fabric-samples/demo-first/workload
```

```
$ mkdir sdk-peer0-org1
```

```
$ mkdir sdk-peer0-org2
```

```
$ mkdir sdk-peer0-org3
```

```
$ mkdir sdk-peer0-org4
```

```
$ mkdir sdk-peer0-org5
```

```
$ mkdir sdk-peer0-org6
```

```
$ mkdir sdk-peer0-org7
```

```
$ mkdir sdk-peer0-org8
```

```
$ vi invoke.js
```

```
$ vi prepare\_sdk\_peer0\_org1.sh
```

```
$ vi prepare\_sdk\_peer0\_org2.sh
```

```
$ vi prepare_sdk_peer0_org3.sh
```

```
$ vi prepare_sdk_peer0_org4.sh
```

```
$ vi prepare_sdk_peer0_org5.sh
```

```
$ vi prepare_sdk_peer0_org6.sh
```

```
$ vi prepare_sdk_peer0_org7.sh
```

```
$ vi prepare_sdk_peer0_org8.sh
```

```
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ python prepare_wallet_user1_org3.py
$ python prepare_wallet_user1_org4.py
$ python prepare_wallet_user1_org5.py
$ python prepare_wallet_user1_org6.py
$ python prepare_wallet_user1_org7.py
$ python prepare_wallet_user1_org8.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines
```

```
$ ./ssh.sh > result/benchlog
```

```
$ ls result -l
```

```
root@i27xviqssiy2jdsfcldbvz:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@i27xviqssiy2jdsfcldbvz:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
```

```
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
```

```
$ cd $HOME/fabric-samples/demo-first/workload/result
```

```
$ python readthroughput_p1.py // rate of sending endorsed txs
```

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phase_1:
5.7471256178
```

\$ sudo apt-get install python-numpy
\$ python [readdelay_p1.py](#) // delay of endorsing phase

```
root@i27xviqssiy2jdjhqk0lpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002090083
```

// Orderer analyzes the log
\$ cd \$HOME/fabric-samples/demo-first/workload/result
\$ sudo apt-get install python-numpy
//\$ docker logs containername >& ordererlog
\$ **docker logs \$(docker ps -a | grep 'orderer' | awk '{print \$1 }') >& ordererlog**
// \$ scp ordererlog [chwang@1xx.1xx.7xx.1xx:/home/chwang](#)
\$ python [readthroughput_p2.py](#) // rate of generating blocks

```
root@i27xviqssiy2jdsfcldp2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
\$ cd \$HOME/fabric-samples/demo-first/workload/result
\$ sudo apt-get install python-numpy
//\$ docker logs containername >& peerlog
\$ **docker logs \$(docker ps -a | grep 'peer node start' | awk '{print \$1 }') >& peerlog**
// \$ scp peerlog [chwang@1xx.1xx.7xx.1xx:/home/chwang](#)
\$ python2 [readthroughput_p3.py](#)

```
root@i27xviqssiy2jdsfcldpvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// Get block size


```
# peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile  
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/or  
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first  
$ ./clean.sh
```